

# Plataforma de Simulação GenESys

Guia prático do usuário e  
do desenvolvedor GenESys



Prof. Rafael Luiz Cancian, Dr. Eng.

Copyright © 2026 Rafael Luiz Cancian

[www.ine.ufsc.br/rafael.cancian](http://www.ine.ufsc.br/rafael.cancian)

**TODOS OS DIREITOS RESERVADOS – É PROIBIDA A REPRODUÇÃO TOTAL OU PARCIAL, DE QUALQUER FORMA OU POR QUALQUER MEIO.**

Esta é uma obra literária e científica protegida pela LEI Nº 9.610, de 19 de fevereiro de 1998, em seu Capítulo I (Das Obras Protegidas), em seu Art. 7: São obras intelectuais protegidas as criações do espírito, expressas por qualquer meio ou fixadas em qualquer suporte, tangível ou intangível, conhecido ou que se invente no futuro, tais como: I - os textos de obras literárias, artísticas ou científicas; em seu Art. 18: A proteção aos direitos de que trata esta Lei independe de registro; e em seu Art. 29: Depende de autorização prévia e expressa do autor a utilização da obra, por quaisquer modalidades, tais como: I - a reprodução parcial ou integral; II - a edição; III - a adaptação e quaisquer outras transformações; VI - a distribuição para uso ou exploração da obra; IX - a inclusão em base de dados, o armazenamento em computador e as demais formas de arquivamento do gênero; X - quaisquer outras modalidades de utilização existentes ou que venham a ser inventadas.

O Código Penal Brasileiro determina, no artigo 184:

**DOS CRIMES CONTRA A PROPRIEDADE INTELECTUAL.**

Art 184. Violar direito autoral: Pena - detenção de três meses a um ano, ou multa.

§1º - Se a violação consistir na reprodução, por qualquer meio, de obra intelectual, no todo ou em parte, para fins de comércio, sem autorização expressa do autor ou de quem o represente, ou consistir na reprodução de fonograma e videofonograma, sem autorização do produtor ou de quem o represente: Pena - reclusão de um a quatro anos e multa.

**ESTA OBRA NÃO DEVE SER PUBLICADA, REPRODUZIDA, EDITADA, ADAPTADA, ARMAZENADA OU DISTRIBUÍDA, TOTAL OU PARCIALMENTE.** ESTA OBRA, AINDA EM VERSÃO RASCUNHO, FOI DISPONIBILIZADA GRATUITAMENTE PELO AUTOR PARA CONSULTA EXCLUSIVA DOS ALUNOS REGULAMENTE MATRICULADOS NA DISCIPLINA DE MODELAGEM E SIMULAÇÃO (INE5425) DA UNIVERSIDADE FEDERAL DE SANTA CATARINA PARA FINS ESTRITAMENTE EDUCACIONAIS. QUALQUER REPRODUÇÃO, EDIÇÃO, ADAPTAÇÃO, ARMAZENAMENTO OU DISPONIBILIZAÇÃO DESTA OBRA POR QUALQUER MEIO CONSTITUI VIOLAÇÃO DOS DIREITOS AUTORAIS E ESTÁ SUJEITA ÀS PENALIDADES PREVISTAS EM LEI.

*Março de 2026*



# Sumário

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| <b>1</b> | <b>O que é o GenESyS</b>                        | <b>11</b> |
| 1.1      | Introdução                                      | 11        |
| 1.2      | Finalidade do GenESyS                           | 12        |
| 1.3      | Como o usuário trabalha com o GenESyS           | 13        |
| 1.4      | A interface gráfica como ponto principal de uso | 14        |
| 1.5      | Modelos de exemplo como porta de entrada        | 15        |
| 1.6      | O que o leitor encontrará neste manual          | 15        |
| 1.7      | Como ler este livro da forma mais proveitosa    | 16        |
| 1.8      | Considerações Finais                            | 16        |

|          |                                                        |           |
|----------|--------------------------------------------------------|-----------|
| <b>2</b> | <b>Download, instalação e compilação .....</b>         | <b>19</b> |
| 2.1      | Introdução                                             | 19        |
| 2.2      | Obtendo o projeto                                      | 20        |
| 2.3      | O que o usuário realmente precisa nessa fase           | 20        |
| 2.4      | Dependências do ambiente                               | 21        |
| 2.5      | Compilando a aplicação gráfica                         | 21        |
| 2.6      | Como interpretar a configuração e a compilação         | 22        |
| 2.7      | O que fazer em caso de erro                            | 23        |
| 2.8      | Primeira execução da GUI                               | 23        |
| 2.9      | Primeiras observações úteis sobre a interface          | 24        |
| 2.10     | Preparando o próximo passo                             | 24        |
| 2.11     | Considerações Finais                                   | 24        |
| <b>3</b> | <b>Primeiros passos na interface gráfica .....</b>     | <b>27</b> |
| 3.1      | Introdução                                             | 27        |
| 3.2      | A janela principal como ambiente de trabalho           | 28        |
| 3.3      | Abertura da aplicação e primeira leitura visual        | 28        |
| 3.4      | Grupos funcionais da interface                         | 28        |
| 3.5      | A área gráfica do modelo                               | 30        |
| 3.6      | Árvores, painéis e editores auxiliares                 | 31        |
| 3.7      | Fluxo mínimo de orientação do usuário na GUI           | 32        |
| 3.8      | O que não é necessário dominar agora                   | 32        |
| 3.9      | Boas práticas no primeiro contato com a interface      | 33        |
| 3.10     | Considerações Finais                                   | 33        |
| <b>4</b> | <b>Modelos de exemplo e uso inicial do simulador .</b> | <b>35</b> |
| 4.1      | Introdução                                             | 35        |
| 4.2      | Por que começar por modelos de exemplo                 | 36        |

|     |                                                      |    |
|-----|------------------------------------------------------|----|
| 4.3 | A pasta <code>models/</code> como referência de uso  | 37 |
| 4.4 | Abrindo um modelo de exemplo na GUI                  | 38 |
| 4.5 | Como ler um modelo na área gráfica                   | 39 |
| 4.6 | Executando o modelo pela primeira vez                | 40 |
| 4.7 | Modificando um modelo de exemplo de forma controlada | 41 |
| 4.8 | Boas práticas para explorar modelos de exemplo       | 42 |
| 4.9 | Considerações Finais                                 | 43 |

## 5 GUI, Simulang, execução e observação do modelo 45

|     |                                                            |    |
|-----|------------------------------------------------------------|----|
| 5.1 | Introdução                                                 | 45 |
| 5.2 | Da estrutura estática à dinâmica do modelo                 | 46 |
| 5.3 | Comandos de execução e sua lógica de uso                   | 46 |
| 5.4 | Observando o comportamento do modelo durante a execução    | 48 |
| 5.5 | A aba <i>Simulang</i> como representação textual do modelo | 48 |
| 5.6 | Tracing, breakpoints e observabilidade inicial             | 49 |
| 5.7 | Primeira leitura dos resultados                            | 50 |
| 5.8 | Boas práticas para observar modelos em execução            | 51 |
| 5.9 | Considerações Finais                                       | 52 |

## 6 Organização do código-fonte e arquitetura geral 53

|     |                                                               |    |
|-----|---------------------------------------------------------------|----|
| 6.1 | Introdução                                                    | 53 |
| 6.2 | Do simulador como aplicação ao simulador como software        | 54 |
| 6.3 | A lógica geral da árvore de diretórios                        | 54 |
| 6.4 | As grandes áreas de <code>source/</code>                      | 55 |
| 6.5 | Uma leitura em camadas do sistema                             | 57 |
| 6.6 | O papel da classe <code>Simulator</code> na visão de conjunto | 57 |
| 6.7 | Leituras erradas que este capítulo evita                      | 58 |

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| 6.8      | Estratégia recomendada de leitura do código                        | 58        |
| 6.9      | Considerações Finais                                               | 59        |
| <b>7</b> | <b>Núcleo do simulador</b>                                         | <b>61</b> |
| 7.1      | Introdução                                                         | 61        |
| 7.2      | O kernel como infraestrutura de simulação                          | 62        |
| 7.3      | A classe <code>Simulator</code> como ponto de entrada do kernel    | 62        |
| 7.4      | A classe <code>Model</code> como centro do modelo de simulação     | 63        |
| 7.5      | A classe <code>ModelSimulation</code>                              | 64        |
| 7.6      | Eventos, evento corrente e processamento sequencial                | 65        |
| 7.7      | A classe <code>OnEventManager</code> e a observabilidade do núcleo | 66        |
| 7.8      | Breakpoints e controle fino da execução                            | 67        |
| 7.9      | Uma leitura integrada do núcleo                                    | 67        |
| 7.10     | Considerações Finais                                               | 67        |
| <b>8</b> | <b>Componentes, <i>model data</i> e sistema de <i>plugins</i></b>  | <b>69</b> |
| 8.1      | Introdução                                                         | 69        |
| 8.2      | <code>ModelDataDefinition</code> como classe-base do modelo        | 70        |
| 8.3      | <code>ModelComponent</code> como especialização comportamental     | 71        |
| 8.4      | Fluxo principal e dados de apoio                                   | 72        |
| 8.5      | Composição, agregação e o desenho interno do modelo                | 73        |
| 8.6      | O sistema de <i>plugins</i>                                        | 74        |
| 8.7      | Como pensar uma nova extensão                                      | 75        |
| 8.8      | Leitura errada que este capítulo evita                             | 75        |
| 8.9      | Considerações Finais                                               | 75        |
| <b>9</b> | <b>Parser, expressões e linguagem interna</b>                      | <b>77</b> |
| 9.1      | Introdução                                                         | 77        |
| 9.2      | O papel do parser no GenESyS                                       | 78        |

|      |                                                      |    |
|------|------------------------------------------------------|----|
| 9.3  | A interface <code>Parser_if</code>                   | 78 |
| 9.4  | A classe <code>Model</code> como usuária do parser   | 79 |
| 9.5  | A gramática Bison/Flex do GenESyS                    | 80 |
| 9.6  | Símbolos do modelo e dependência do estado corrente  | 81 |
| 9.7  | Pontos de extensão por <i>plugins</i>                | 82 |
| 9.8  | A classe <code>ParserManager</code>                  | 82 |
| 9.9  | Erros, mensagens e robustez                          | 83 |
| 9.10 | Como estudar e modificar o parser de forma produtiva | 83 |
| 9.11 | Considerações Finais                                 | 83 |

## 10 Applications, tools, exemplos em C++, testes e evolução do projeto ..... 85

|      |                                                   |    |
|------|---------------------------------------------------|----|
| 10.1 | Introdução                                        | 85 |
| 10.2 | Applications como projeções do kernel             | 86 |
| 10.3 | Os exemplos em C++ como material de desenvolvedor | 87 |
| 10.4 | Tools como camada de apoio                        | 88 |
| 10.5 | A importância dos testes                          | 88 |
| 10.6 | Build, aplicações e seleção de comportamentos     | 89 |
| 10.7 | Evolução do projeto e caminhos de contribuição    | 90 |
| 10.8 | Uma visão final integrada do GenESyS              | 91 |
| 10.9 | Considerações Finais                              | 91 |

## Bibliografia ..... 93

RASCUNHO



## Prefácio

Este livro apresenta o simulador *GenESyS* como ferramenta de uso prático e como plataforma de desenvolvimento. Seu objetivo é duplo. Em primeiro lugar, servir como manual para o usuário que deseja instalar o sistema, compreender suas formas de utilização, construir modelos, executá-los e interpretar seus resultados. Em segundo lugar, servir como manual para o desenvolvedor que deseja entender a arquitetura interna do simulador e estendê-lo por meio de novos componentes, novas *data definitions*, novas funções de linguagem, novas ferramentas e novas aplicações.

Diferentemente de um texto amplo sobre modelagem e simulação de sistemas, este livro é deliberadamente centrado no *software*. Conceitos gerais aparecem apenas quando são necessários para esclarecer o funcionamento do *GenESyS*, sua organização interna ou a forma correta de utilizar algum de seus recursos. Com isso, o foco permanece naquilo que realmente interessa ao leitor deste volume: como o simulador funciona, como ele é usado e como pode ser evoluído.

O livro foi organizado com uma progressão simples. Os capítulos iniciais tratam do *GenESyS* como ferramenta de uso direto, apresentando instalação, compilação, execução e os modos mais importantes de interação com o simulador. Em seguida, o texto avança para o uso programático da biblioteca e para a estrutura essencial do modelo e do núcleo de simulação.

Por fim, o livro passa a focar o *GenESyS* como plataforma extensível, cobrindo os elementos centrais de desenvolvimento, como novos componentes, *plugins*, parser, ferramentas e aplicações.

Essa organização procura atender a dois perfis de leitor sem fragmentar excessivamente o material. O primeiro é o leitor que deseja apenas utilizar o simulador. O segundo é o leitor que pretende colaborar com o projeto ou adaptá-lo a novas necessidades. Embora esses perfis tenham objetivos distintos, ambos se beneficiam de uma compreensão clara da relação entre uso externo e estrutura interna do sistema. Por essa razão, este livro foi escrito de modo a permitir leitura progressiva: quem deseja apenas usar o simulador pode concentrar-se inicialmente nos capítulos de entrada e uso; quem deseja desenvolvê-lo pode avançar para os capítulos finais, voltados à arquitetura e à extensão.

Também é importante destacar que este volume privilegia aspectos concretos do projeto. Por isso, o texto utiliza exemplos reais, trechos reais de código-fonte, diagramas estruturais e descrições diretamente associadas aos pacotes, classes e mecanismos efetivamente presentes no *GenESyS*. Sempre que possível, evita-se a duplicação de explicações teóricas longas, priorizando-se descrições operacionais, decisões arquiteturais e procedimentos de uso e desenvolvimento.

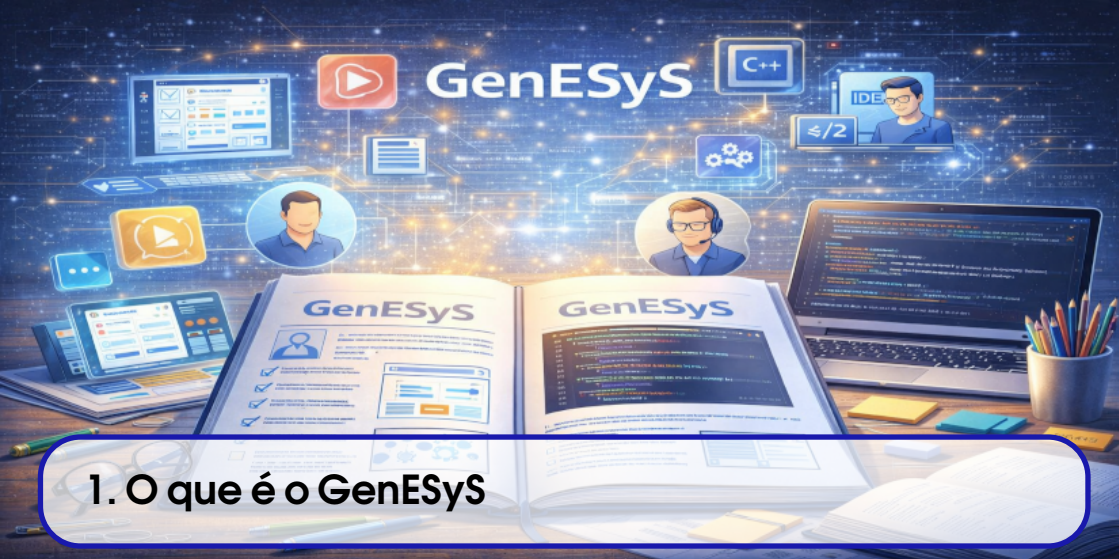
O *GenESyS* é tratado aqui não apenas como simulador, mas como infraestrutura aberta à evolução. Essa característica influencia fortemente o conteúdo do livro. Ao longo dos capítulos, o leitor perceberá que diversas partes do sistema foram concebidas para permitir ampliação progressiva, como o mecanismo de *plugins*, a separação entre componentes e *data definitions*, a organização do kernel, a linguagem interna do simulador e a infraestrutura de parser. Desse modo, o manual não se limita a explicar como operar um sistema pronto, mas também como participar de sua continuidade técnica.

Por fim, este livro foi escrito com uma orientação explicitamente prática. Ele não pretende esgotar todos os detalhes possíveis do projeto nem substituir documentação complementar de referência fina, como código-fonte, wiki, documentação do repositório ou comentários internos do sistema. Seu papel é oferecer ao leitor um caminho técnico curto, consistente e suficientemente profundo para que ele possa usar o *GenESyS* com segurança e, quando necessário, atuar sobre sua implementação.

Espera-se, assim, que este volume seja útil tanto para quem está dando os primeiros passos com o simulador quanto para quem deseja compreendê-lo como arquitetura de software e plataforma de desenvolvimento. Em ambos os casos, o ponto central permanece o mesmo: transformar o *GenESyS* em um instrumento efetivo de trabalho, estudo, experimentação e evolução tecnológica.

Deve-se observar, entretanto, que esta obra permanece em construção e aperfeiçoamento contínuo. Capítulos, exemplos, figuras, exercícios e anexos seguem sendo revisados, condensados e refinados editorialmente. Por essa razão, este material não deve ser considerado versão finalizada ou estabilizada. Assim, ele não pode ser publicado, reproduzido, editado, adaptado, armazenado ou distribuído, total ou parcialmente.

Florianópolis, Março de 2026.



# 1. O que é o GenESyS

## 1.1 Introdução

O *GenESyS* é um simulador de sistemas concebido para servir, ao mesmo tempo, como ferramenta prática de modelagem e simulação e como base de software para expansão e pesquisa. Neste livro, porém, essas duas dimensões não serão tratadas de maneira simultânea. A ordem foi deliberadamente invertida em favor do uso. Primeiro, o leitor aprenderá a trabalhar com o *GenESyS* como usuário do simulador. Depois, já familiarizado com a aplicação e com os modelos, passará a estudá-lo como sistema de software e como plataforma extensível.

Essa escolha é importante. Um erro comum ao abordar um simulador academicamente rico é começar pelo que ele tem de mais interno: classes, subsistemas, parser, *plugins*, infraestrutura de execução. Embora tudo isso seja essencial, essa não é a melhor porta de entrada para quem deseja realmente colocar o sistema para funcionar. O caminho mais produtivo começa pelo uso: compreender o que o simulador oferece, como o usuário o enxerga, como os modelos são abertos, executados e observados, e como a interface organiza esse trabalho.

Este capítulo, portanto, cumpre uma função de orientação. Ele apresenta o *GenESyS* como ambiente de trabalho para modelagem e simulação, define o papel da interface gráfica como ponto principal de interação do usuário e explica como o restante do manual foi orga-

nizado. O objetivo não é ainda ensinar a instalar o sistema nem usar seus menus em detalhe, mas estabelecer uma visão suficientemente clara para que os próximos capítulos façam sentido como continuação natural.

## 1.2 Finalidade do GenESyS

O *GenESyS* deve ser entendido, em primeiro lugar, como um ambiente para trabalhar com modelos de simulação. Isso significa que seu valor para o usuário não está apenas em “rodar” um programa, mas em permitir um ciclo completo de trabalho com modelos: abrir, examinar, parametrizar, executar, observar, comparar, registrar e aperfeiçoar.

Do ponto de vista prático, esse ambiente precisa dar suporte a um conjunto de atividades que o usuário realiza recorrentemente:

- carregar modelos já existentes;
- inspecionar sua estrutura;
- alterar parâmetros e propriedades;
- executar a simulação;
- observar a dinâmica do modelo;
- interpretar resultados iniciais;
- salvar versões modificadas do trabalho.

O *GenESyS* foi pensado exatamente para esse tipo de uso. Em vez de ser apenas um programa isolado, ele se apresenta como um espaço de trabalho onde o modelo é o objeto central da interação. O usuário não lida apenas com comandos abstratos, mas com entidades concretas do domínio da simulação: componentes, fluxos, dados de apoio, tempo simulado, eventos, resultados e representações complementares do mesmo modelo.

### 1.2.1 GenESyS como ferramenta de modelagem e simulação

Quando visto pela ótica do usuário, o *GenESyS* é uma ferramenta de modelagem e simulação. Isso quer dizer que a pergunta principal deixa de ser “como o software foi implementado?” e passa a ser “como o modelo é construído, executado e observado dentro do software?”.

Essa mudança de perspectiva é essencial para a primeira parte deste manual. O usuário quer saber:

- como obter o simulador;
- como colocá-lo em funcionamento;
- como abrir modelos de exemplo;
- como usar a interface gráfica;
- como executar simulações;
- como entender o que o sistema está mostrando.

É exatamente esse percurso que será seguido nos primeiros capítulos.

1.2.2 GenESyS como sistema em evolução

Embora este capítulo trate o *GenESyS* principalmente como ferramenta de uso, convém registrar desde já que ele também é um projeto de software em evolução. Isso significa que, por trás da aplicação gráfica e dos modelos que o usuário manipula, há uma infraestrutura interna composta por kernel, parser, *plugins*, aplicações auxiliares e mecanismos de teste.

Essa dimensão, contudo, não deve interferir no caminho inicial do leitor. O fato de o sistema também poder ser estudado e aperfeiçoado como software não altera a ordem de aprendizagem proposta neste livro. Primeiro, o leitor usará o simulador. Depois, estudará sua implementação.

1.3 Como o usuário trabalha com o GenESyS

O trabalho do usuário com o *GenESyS* pode ser descrito como um fluxo relativamente claro, embora composto por várias etapas complementares. Em uma situação típica, o usuário:

- 1. obtém o projeto e prepara o ambiente;
- 2. compila e inicia a aplicação;
- 3. abre a interface gráfica;
- 4. carrega um modelo já salvo ou inicia um novo;
- 5. inspeciona os elementos do modelo;
- 6. executa a simulação;
- 7. observa seu comportamento e seus resultados;
- 8. salva o trabalho e suas variações.

Esse fluxo é mais importante do que pode parecer à primeira vista. Ele define não apenas a forma como o usuário trabalha com o simulador, mas também a organização da primeira metade do livro. Cada um dos próximos capítulos aprofundará uma dessas etapas.

Tabela 1.1: Fluxo geral de trabalho do usuário no GenESyS.

| Etapas                 | Objetivo prático                                               |
|------------------------|----------------------------------------------------------------|
| Obter o sistema        | Ter acesso ao código, às dependências e ao ambiente necessário |
| Compilar e iniciar     | Colocar a aplicação gráfica em funcionamento                   |
| Abrir um modelo        | Trabalhar sobre um caso concreto dentro do simulador           |
| Explorar a GUI         | Entender como a interface organiza o modelo e a simulação      |
| Executar o modelo      | Observar a dinâmica simulada do sistema                        |
| Inspecionar resultados | Ler sinais iniciais do comportamento e do desempenho do modelo |
| Salvar e continuar     | Preservar o trabalho e preparar novas iterações                |

O aspecto mais importante aqui é que o *GenESyS* não deve ser usado como um sistema de comandos isolados, mas como um ambiente de trabalho iterativo. O usuário abre modelos,

explora, modifica, executa, compara e reexecuta. É essa natureza iterativa que faz a GUI ser tão importante no uso inicial da ferramenta.

## 1.4 A interface gráfica como ponto principal de uso

Nesta primeira parte do manual, a interface gráfica será tratada como o caminho principal de uso do *GenESyS*. Essa decisão não é apenas editorial; ela decorre do perfil de uso esperado para o usuário do simulador. Quem está trabalhando com modelos, especialmente em fase inicial de aprendizagem, precisa de uma forma de interação mais direta, mais visível e mais guiada do que a mera manipulação de artefatos internos do projeto.

A GUI cumpre precisamente esse papel. Ela organiza o modelo como objeto visual e operacional. Por meio dela, o usuário consegue:

- abrir modelos salvos;
- observar sua estrutura;
- navegar por elementos e propriedades;
- acionar comandos de simulação;
- acompanhar o comportamento do sistema;
- acessar visões complementares do modelo, inclusive sua descrição textual.

É por isso que este livro não começará por aplicações de terminal nem por exemplos implementados em C++. Esses artefatos são importantes, mas pertencem ao campo do desenvolvedor. Para o usuário, o caminho mais natural é a interface gráfica e os modelos já salvos.

### 1.4.1 Por que a GUI vem antes da linguagem

Outra decisão importante é a ordem entre GUI e linguagem. A linguagem do *GenESyS*, apresentada ao usuário por meio da aba *Simulang*, só fará sentido depois que o leitor já tiver desenvolvido uma noção concreta do modelo na interface gráfica. Se o texto da linguagem for introduzido cedo demais, ele tende a ser percebido como um formalismo opaco. Em contrapartida, depois que o usuário já viu o modelo na GUI, executou sua simulação e observou seus elementos, a descrição textual passa a funcionar como um segundo modo de leitura do mesmo sistema.

Essa ordem é pedagogicamente superior:

1. primeiro, o usuário vê e manipula o modelo;
2. depois, ele observa como esse modelo também pode ser descrito textualmente.

Assim, a linguagem entra como aprofundamento e não como obstáculo inicial.

## 1.5 Modelos de exemplo como porta de entrada

O contato inicial com um simulador raramente é mais produtivo quando se começa do zero. Para a maioria dos usuários, a melhor forma de entrar no sistema é abrir um modelo já existente, observar como ele está organizado e usar esse caso como base para exploração. O *GenESyS* segue exatamente essa lógica.

Os modelos de exemplo desempenham, aqui, um papel didático e operacional ao mesmo tempo. Eles são didáticos porque ajudam o usuário a enxergar casos concretos de uso da ferramenta. E são operacionais porque já podem ser abertos, executados, inspecionados e modificados dentro da GUI.

Ao começar por um modelo salvo, o usuário não precisa enfrentar simultaneamente todas as dificuldades iniciais:

- aprender a interface;
- escolher componentes;
- definir parâmetros;
- organizar o fluxo;
- verificar se o modelo está coerente.

Em vez disso, ele passa a trabalhar com um modelo que já existe e pode concentrar sua atenção em compreender o simulador. Essa será a estratégia adotada nos próximos capítulos.

## 1.6 O que o leitor encontrará neste manual

Este livro foi organizado em duas partes claramente distintas.

A primeira parte é o *manual do usuário*. Nela, o leitor aprenderá a:

- obter o projeto;
- preparar o ambiente;
- compilar a aplicação;
- abrir a GUI;
- trabalhar com modelos de exemplo;
- executar simulações;
- observar resultados e representações do modelo.

A segunda parte é o *manual do desenvolvedor*. Nela, o foco se deslocará da operação do simulador para sua estrutura interna. O leitor passará a estudar:

- a organização do código-fonte;
- o kernel do simulador;
- componentes e *data definitions*;
- o sistema de *plugins*;
- o parser e a linguagem interna;
- aplicações auxiliares, ferramentas e testes.

Tabela 1.2: Estrutura geral do manual do GenESyS.

| Parte do livro          | Finalidade principal                                                                                     |
|-------------------------|----------------------------------------------------------------------------------------------------------|
| Manual do Usuário       | Ensinar a usar o GenESyS como aplicação de modelagem e simulação, com ênfase na GUI e nos modelos salvos |
| Manual do Desenvolvedor | Ensinar a compreender, estender e aperfeiçoar o GenESyS como projeto de software                         |

Essa divisão não deve ser vista como mera conveniência editorial. Ela corresponde a duas formas reais de trabalhar com o sistema. Primeiro, o leitor precisa conseguir usar o simulador. Depois, se desejar, poderá compreender e modificar sua implementação.

## 1.7 Como ler este livro da forma mais proveitosa

Há pelo menos dois perfis principais de leitor para este manual.

O primeiro é o leitor que deseja apenas usar o *GenESyS* como simulador. Nesse caso, a leitura natural é linear ao longo da primeira parte do livro, com atenção especial aos capítulos sobre instalação, GUI, modelos de exemplo, execução e observação dos resultados.

O segundo é o leitor que, além de usar o sistema, pretende estudá-lo e aperfeiçoá-lo como desenvolvedor. Para esse perfil, a melhor estratégia é primeiro percorrer também a parte de usuário. Isso evita que a arquitetura interna seja estudada de forma abstrata demais. Um desenvolvedor que já viu a GUI em funcionamento e já abriu modelos reais compreende melhor, depois, o papel do kernel, do parser e dos *plugins*.

Em outras palavras, a primeira parte do livro não é apenas preparatória para o usuário; ela também prepara o futuro desenvolvedor para ler o sistema do ponto de vista correto.

## 1.8 Considerações Finais

Este capítulo apresentou o *GenESyS* como ferramenta de modelagem e simulação, enfatizando a perspectiva do usuário. O ponto mais importante a reter é que o sistema deve ser inicialmente compreendido como ambiente de trabalho e não como objeto de análise arquitetural. Para o usuário, o caminho natural começa pela interface gráfica, pelos modelos salvos e pela execução de casos concretos. Só depois faz sentido aprofundar-se na linguagem textual do modelo e, mais adiante, na estrutura interna do software.

Essa ordem não reduz a riqueza do *GenESyS*; pelo contrário, ela a torna mais acessível. Ao começar pelo uso, o leitor constrói uma base concreta para tudo o que virá depois. Com isso estabelecido, o próximo passo natural é preparar o ambiente de trabalho, isto é, obter o projeto, instalar as dependências, compilar a aplicação e alcançar a primeira execução funcional da GUI.

Teste seu entendimento desse conteúdo respondendo as seguintes perguntas:

1. Por que este livro começa tratando o *GenESyS* como ferramenta de uso antes de tratá-lo como projeto de software?
2. Qual é a vantagem de adotar a GUI como ponto principal de entrada para o usuário?
3. Por que a aba *Simulang* e a linguagem do modelo devem ser apresentadas apenas depois da interface gráfica?
4. Por que os modelos de exemplo são a melhor porta de entrada para o primeiro contato prático com o simulador?

RASCUNHO



## 2. Download, instalação e compilação

### 2.1 Introdução

Depois de compreender o papel do *GenESyS* e a forma como este manual foi organizado, o leitor precisa colocá-lo em funcionamento. Esse é o objetivo deste capítulo. Em vez de tratar a compilação como tema central do livro, ela será tratada aqui como aquilo que realmente é para o usuário: uma etapa necessária para alcançar a aplicação gráfica e começar a trabalhar com modelos.

É importante notar que o *GenESyS* é um projeto de software mais amplo do que uma única aplicação. Há kernel, parser, *plugins*, testes, aplicações auxiliares e outros elementos internos. No entanto, essa complexidade não precisa ser absorvida pelo usuário logo no início. A meta prática desta etapa é mais simples: obter o projeto, preparar o ambiente, compilar a GUI e validar a primeira execução da aplicação gráfica. Se isso for alcançado com segurança, o leitor já terá o necessário para avançar para a fase de uso efetivo do simulador.

## 2.2 Obtendo o projeto

O *GenESyS* está disponível como repositório de código-fonte. Assim, o primeiro passo consiste em obter uma cópia local do projeto. Em um ambiente com *git* devidamente instalado, esse procedimento é direto.

```
1 git clone https://github.com/rlcancian/Genesys-Simulator.git
2 cd Genesys-Simulator
```

Listing 2.1: Clonagem inicial do repositório do GenESyS

Esse comando produz uma cópia local do repositório. A partir daí, o usuário passa a ter em sua máquina o conjunto de arquivos necessários para preparar o ambiente de trabalho. Convém, nesse ponto, fazer uma inspeção inicial da raiz do projeto, apenas para reconhecer sua presença e seu volume estrutural.

```
1 pwd
2 ls
```

Listing 2.2: Inspeção inicial da raiz do repositório

O usuário não precisa ainda interpretar em profundidade tudo o que aparece nessa listagem. Basta reconhecer que há uma árvore-fonte, há uma pasta de modelos e há arquivos de configuração que sustentam a compilação e o restante do projeto.

## 2.3 O que o usuário realmente precisa nessa fase

Uma dificuldade frequente em projetos acadêmicos e de pesquisa é supor que o usuário precise compreender toda a organização interna antes de conseguir utilizá-los. Para o *GenESyS*, essa estratégia seria ruim. O usuário desta primeira parte do livro não precisa estudar imediatamente a estrutura do kernel, os *plugins* ou o parser. Ele precisa apenas do que for suficiente para:

- preparar um ambiente coerente;
- compilar a aplicação gráfica;
- iniciar a GUI;
- confirmar que a interface está funcional.

Portanto, a organização do repositório deve ser lida, neste momento, apenas com uma preocupação prática. A pasta *source/* contém o código da aplicação e do sistema. A pasta *models/* será importante nos próximos capítulos, pois é dela que virão os modelos de exemplo usados pelo usuário. Demais elementos do projeto podem permanecer, por ora, em segundo plano.

## 2.4 Dependências do ambiente

A versão atual do *GenESyS* utiliza *CMake* como sistema principal de configuração e build. Isso significa que o ambiente de compilação do usuário precisa oferecer um conjunto mínimo de ferramentas compatíveis com essa organização. Além do próprio *CMake*, o projeto depende de um compilador C++ moderno e de uma instalação funcional de Qt para a aplicação gráfica.

Do ponto de vista prático, convém pensar o ambiente em três camadas:

1. ferramentas de build;
2. bibliotecas gráficas e de apoio;
3. compilador e utilitários gerais de desenvolvimento.

Na primeira camada, o elemento central é o *CMake*. Na segunda, o ponto mais importante é o Qt, especialmente os módulos associados à interface gráfica. Na terceira, entram o compilador C++ e as ferramentas usuais de desenvolvimento da plataforma escolhida.

No caso específico da GUI, o projeto procura preferencialmente por Qt6 e, se necessário, admite fallback para Qt5. Para o usuário, isso significa que o ambiente precisa disponibilizar ao menos uma dessas instalações de forma coerente.

### 2.4.1 Checklist mínimo do ambiente

Antes de partir para a configuração do build, o usuário deve confirmar que possui, no mínimo:

- git;
- cmake;
- um compilador C++ moderno;
- uma instalação funcional de Qt;
- ferramentas básicas de terminal e sistema de arquivos.

Esse checklist não substitui instruções específicas de cada plataforma, mas evita que o processo de compilação seja iniciado em ambiente claramente incompleto.

## 2.5 Compilando a aplicação gráfica

A compilação da GUI deve ser conduzida em um diretório de build separado da árvore-fonte. Essa prática é recomendável por duas razões. A primeira é manter o repositório limpo, sem misturar arquivos gerados com os arquivos originais do projeto. A segunda é tornar mais fácil a repetição do processo em caso de ajustes, limpeza ou reconfiguração do ambiente.

```
1 mkdir -p build
2 cd build
```

Listing 2.3: Criação do diretório de build

Com o diretório de build criado, o usuário pode configurar o projeto. Como o caminho

principal desta primeira parte do livro é a GUI, a configuração deve habilitar explicitamente a aplicação gráfica. Uma configuração inicial típica pode ser feita assim:

```
1 cmake .. \  
2 -DGENESYS_BUILD_GUI_APPLICATION=ON
```

Listing 2.4: Configuração inicial do build da GUI

Dependendo da plataforma e do ambiente local, o usuário pode querer ajustar outras opções. No entanto, como orientação inicial do manual, o ponto essencial é garantir que a GUI seja incluída no processo de build. Depois disso, a compilação propriamente dita pode ser realizada.

```
1 cmake --build . --target genesys_qt_gui_application -j
```

Listing 2.5: Compilação da GUI do GenESyS

Em alguns ambientes, o alvo agregado da GUI também pode ser usado, quando disponível:

```
1 cmake --build . --target genesys_gui -j
```

Listing 2.6: Alternativa de compilação pelo alvo agregado da GUI

### 2.5.1 Por que a GUI é explicitamente habilitada

É importante compreender a lógica dessa configuração. O projeto contém mais de uma aplicação possível, mas o manual do usuário não será guiado por todas elas. O alvo desta primeira metade do livro é a interface gráfica. Por isso, a GUI é habilitada explicitamente na configuração do build. Essa decisão reduz ambiguidades e alinha o processo técnico à forma real como o usuário trabalhará com o sistema.

## 2.6 Como interpretar a configuração e a compilação

Em um primeiro contato, a saída textual do CMake e do processo de compilação pode parecer excessivamente detalhada. Contudo, o usuário não precisa interpretar tudo. Há apenas alguns sinais essenciais que devem ser observados.

Na configuração:

- o Qt deve ser localizado corretamente;
- o processo não deve terminar com erro fatal;
- o diretório de build deve ser gerado normalmente.

Na compilação:

- o processo deve completar a geração do executável da GUI;
- não devem aparecer falhas de linkedição ou de bibliotecas ausentes;
- o alvo da aplicação gráfica deve ser produzido ao final.

Essa leitura seletiva da saída do build é importante. O usuário não precisa ainda analisar toda a cadeia interna de alvos do projeto. Ele precisa apenas confirmar que o caminho até a aplicação gráfica foi bem-sucedido.

## 2.7 O que fazer em caso de erro

Problemas de compilação fazem parte do trabalho com software, especialmente quando o projeto está em evolução. Isso não significa, porém, que o usuário precise entrar imediatamente em depuração arquitetural do sistema. A abordagem mais útil é começar pelas causas mais simples.

Em caso de erro, verifique primeiro:

- se o Qt foi instalado corretamente;
- se o cmake está disponível e atualizado;
- se o compilador C++ está corretamente configurado;
- se o diretório de build foi criado limpo;
- se não há resíduos de configurações anteriores incompatíveis.

Uma prática particularmente útil é remover o diretório build e repetir o processo em ambiente limpo quando surgirem inconsistências difíceis de interpretar. Em muitos casos, isso elimina resíduos de configuração ou escolhas anteriores inadequadas.

## 2.8 Primeira execução da GUI

Concluída a compilação, o próximo objetivo é iniciar a aplicação gráfica. O executável gerado corresponde ao alvo `genesys_qt_gui_application`. Em um cenário típico, sua execução inicial será feita diretamente a partir do diretório de build ou do caminho exato em que o gerador utilizado o tiver produzido.

```
1 ./genesys_qt_gui_application
```

Listing 2.7: Exemplo genérico de primeira execução da GUI

O caminho exato pode variar, mas a intenção é sempre a mesma: chegar à janela principal da aplicação e confirmar que a GUI está funcional.

### 2.8.1 O que validar na primeira execução

Ao iniciar a aplicação pela primeira vez, o usuário deve verificar alguns pontos muito concretos:

- a janela principal abre corretamente;
- menus e áreas visíveis da interface aparecem de forma coerente;
- não há falha estrutural imediata ao iniciar a aplicação;
- a GUI parece pronta para abrir ou criar modelos.

Ainda não é necessário abrir modelos ou executar simulações. A meta desta etapa é mais modesta, mas fundamental: confirmar que o ambiente preparado realmente leva à aplicação gráfica em funcionamento.

## 2.9 Primeiras observações úteis sobre a interface

Uma vez que a GUI esteja aberta, o usuário já pode fazer uma leitura muito breve da aplicação, sem ainda tentar usá-la em profundidade. Convém observar:

- a presença de barra de menus;
- a área central de trabalho;
- a existência de painéis, árvores ou abas laterais;
- a disponibilidade de comandos ligados a modelo, edição e simulação.

Essa observação não substitui o capítulo seguinte, que tratará especificamente da interface gráfica. Contudo, ela ajuda a transformar a primeira execução em experiência mais concreta do que simplesmente “o programa abriu”.

## 2.10 Preparando o próximo passo

Se o leitor conseguiu chegar até aqui, então já possui o elemento mais importante desta fase inicial: uma instalação funcional do *GenESyS* pronta para uso. Isso é suficiente para avançar. Não é necessário, neste momento, aprofundar-se nos detalhes da estrutura do código nem explorar todas as opções possíveis de build. A partir daqui, o manual passa do problema técnico da preparação do ambiente para o problema prático do uso do simulador.

Em outras palavras, o leitor já está em condição de começar a trabalhar com a aplicação gráfica propriamente dita.

## 2.11 Considerações Finais

O propósito deste capítulo foi conduzir o leitor desde a obtenção do projeto até a primeira execução funcional da GUI do *GenESyS*. Embora essa etapa envolva download, dependências, configuração e compilação, seu papel dentro do livro continua sendo instrumental. O objetivo não era transformar o build no centro do manual, mas remover a barreira técnica necessária para que o uso do simulador pudesse começar de fato.

Com a aplicação gráfica em funcionamento, o próximo passo natural é compreender melhor a janela principal e sua lógica de organização. Por isso, o capítulo seguinte passa

a tratar da interface gráfica como ambiente de trabalho do usuário, mostrando como a GUI organiza menus, áreas de edição, comandos de simulação, painéis de navegação e diferentes visões do modelo.

Teste seu entendimento desse conteúdo respondendo as seguintes perguntas:

1. Por que a compilação da GUI é tratada neste manual como meio para alcançar o uso do simulador, e não como eixo organizador do livro?
2. Que dependências mínimas o ambiente do usuário precisa fornecer antes da configuração do build?
3. Por que é recomendável compilar o projeto em um diretório de build separado?
4. Que sinais mostram que a primeira execução da aplicação gráfica foi bem-sucedida?

RASCUNHO



## 3. Primeiros passos na interface gráfica

### 3.1 Introdução

Uma vez que a aplicação gráfica do *GenESyS* esteja em funcionamento, o usuário entra, de fato, no simulador. É a partir desse momento que o sistema deixa de ser apenas um projeto compilado e passa a ser um ambiente de trabalho. Este capítulo tem justamente essa função: transformar a primeira execução da GUI em um primeiro contato orientado com a interface.

A proposta aqui não é ainda modelar em detalhe nem executar exemplos completos. Antes disso, o leitor precisa reconhecer a lógica da janela principal, entender quais áreas da interface são mais importantes, perceber onde estão os comandos essenciais e construir uma noção suficientemente clara do espaço de trabalho que o *GenESyS* oferece. Em outras palavras, este capítulo ensina o usuário a se localizar dentro da aplicação.

Essa etapa é mais importante do que pode parecer. Uma interface rica, com múltiplos menus, abas, árvores, painéis e comandos de simulação, pode gerar a impressão de excesso de complexidade quando o usuário tenta compreendê-la de uma só vez. A melhor estratégia é outra: reconhecer primeiro a estrutura global da aplicação, depois identificar seus grupos funcionais e, só então, começar a trabalhar com modelos concretos. É exatamente esse percurso que será seguido aqui.

## 3.2 A janela principal como ambiente de trabalho

A interface gráfica do *GenESyS* foi organizada em torno de uma janela principal que centraliza as operações de modelagem, inspeção, simulação, depuração e relatório. Isso significa que o usuário não está diante de uma aplicação com um único painel ou com um conjunto mínimo de comandos isolados. Ao contrário, a GUI foi concebida como um ambiente de trabalho completo, no qual diferentes tarefas podem ser realizadas por meio de diferentes visões do mesmo modelo.

Do ponto de vista do uso, essa janela principal deve ser compreendida como o espaço onde se articulam cinco dimensões fundamentais do trabalho com o simulador:

1. a gestão do modelo;
2. a edição gráfica;
3. a parametrização e inspeção;
4. a execução da simulação;
5. a observação dos resultados e do comportamento do sistema.

Essa organização é especialmente importante porque o usuário não trabalha com o modelo apenas antes da execução nem apenas depois dela. O modelo é construído, ajustado, inspecionado e reavaliado continuamente. A GUI foi estruturada para sustentar exatamente esse ciclo.

## 3.3 Abertura da aplicação e primeira leitura visual

Ao iniciar a aplicação gráfica, o usuário deve resistir à tentação de clicar imediatamente em todos os menus e comandos disponíveis. O melhor caminho é uma leitura visual orientada da janela principal. Essa leitura pode ser feita em etapas muito simples.

Primeiro, observe a barra de menus e note que ela organiza o trabalho em categorias funcionais. Em geral, estão presentes ações de modelo, edição, visualização, simulação, ferramentas e informações auxiliares. Segundo, observe a área central da interface, que é o espaço principal de representação gráfica do modelo. Terceiro, note a existência de abas, painéis laterais ou inferiores e árvores de navegação, que complementam a visão central. Quarto, observe que a aplicação não foi desenhada apenas para editar um diagrama, mas para acompanhar a vida inteira do modelo, inclusive sua simulação e sua inspeção.

Essa primeira leitura já é suficiente para uma conclusão importante: a GUI do *GenESyS* não é um simples editor gráfico. Ela é uma interface de trabalho integrada.

## 3.4 Grupos funcionais da interface

A forma mais produtiva de compreender a GUI é dividi-la em grupos funcionais. Em vez de decorar comandos isolados, o usuário deve perguntar: que tipo de trabalho cada parte da interface organiza?

### 3.4.1 Comandos de modelo

O primeiro grupo funcional é o dos comandos de modelo. São eles que permitem iniciar um novo trabalho, abrir um modelo já salvo, salvar o modelo atual, fechá-lo, verificar sua consistência e obter informações sobre ele. Esses comandos estruturam o ciclo mais básico de uso da ferramenta.

Na prática, isso significa que o usuário deve reconhecer logo no início onde estão as ações responsáveis por:

- criar um novo modelo;
- abrir um modelo existente;
- salvar o trabalho;
- fechar o modelo atual;
- verificar o modelo antes de simular.

Esses comandos serão usados repetidamente ao longo de toda a primeira parte do livro.

### 3.4.2 Comandos de edição

O segundo grupo é o dos comandos de edição. Aqui entram ações como desfazer, refazer, copiar, colar, excluir, agrupar, desagrupar e, em alguns casos, comandos de organização e alinhamento. Esse conjunto revela que a interface foi pensada para edição efetiva do modelo, e não apenas para observação passiva.

Mesmo que o usuário ainda não esteja construindo modelos novos, é importante perceber desde já que a GUI suporta modificação de elementos gráficos e estruturais. Isso será útil quando os modelos de exemplo começarem a ser alterados, mesmo que de forma leve.

### 3.4.3 Comandos de visualização

Outro grupo importante é o dos comandos de visualização. Eles incluem ajustes como zoom, grade, guias e outros recursos que facilitam a organização visual do espaço gráfico. Em um primeiro contato, o usuário não precisa explorar todos esses recursos em detalhe, mas deve reconhecê-los como ferramentas de apoio à leitura e à edição do modelo.

Em especial, o zoom já costuma ser útil logo no início, porque modelos diferentes podem exigir diferentes escalas de observação.

### 3.4.4 Comandos de simulação

Um grupo central da GUI é formado pelos comandos ligados à execução da simulação. Entre eles estão, tipicamente, iniciar, pausar, retomar, parar, avançar passo a passo e configurar a simulação. Esses comandos mostram que a aplicação não apenas organiza o modelo como estrutura, mas também acompanha sua execução.

Neste capítulo, o leitor deve apenas reconhecer a existência e a localização desses co-

mandos. Seu uso efetivo será aprofundado quando começarmos a trabalhar com modelos concretos. Ainda assim, é importante fixar a ideia de que a mesma GUI usada para abrir e editar o modelo também será usada para conduzir sua simulação.

### 3.4.5 Comandos de inspeção e apoio

Além dos grupos anteriores, a interface também oferece mecanismos de inspeção e apoio: painéis de propriedades, árvores de componentes e *data definitions*, áreas de depuração, abas de relatórios e outras visões auxiliares. A presença desses recursos reforça um ponto essencial: o *GenESyS* não foi desenhado apenas para desenhar modelos, mas para estudá-los em execução.

Tabela 3.1: Grupos funcionais principais da GUI do GenESyS.

| Grupo funcional  | Papel no trabalho do usuário                                                    |
|------------------|---------------------------------------------------------------------------------|
| Modelo           | Criar, abrir, salvar, fechar, verificar e organizar o trabalho com modelos      |
| Edição           | Alterar e reorganizar elementos do modelo                                       |
| Visualização     | Ajustar escala, guias e organização visual da área gráfica                      |
| Simulação        | Iniciar, pausar, retomar, interromper e acompanhar a execução                   |
| Inspeção e apoio | Examinar propriedades, elementos do modelo, resultados e informações auxiliares |

## 3.5 A área gráfica do modelo

A área gráfica é o centro visual do trabalho com o *GenESyS*. É nela que o modelo aparece de forma diagramática e é nela que a maior parte da atenção do usuário tende a se concentrar durante a leitura e a edição dos modelos. Em um primeiro contato, essa área deve ser compreendida como o espaço principal onde o fluxo do modelo se torna visível.

Isso é importante porque o usuário precisa começar a ler o simulador pela forma como ele mostra o modelo, e não apenas pela forma como o programa organiza seus menus. A área gráfica permite visualizar:

- os componentes presentes no modelo;
- as conexões entre esses componentes;
- a lógica geral do fluxo modelado;
- a distribuição visual dos elementos dentro do espaço de trabalho.

Mesmo antes de abrir um modelo concreto, o usuário já deve perceber essa área como o núcleo da experiência visual do simulador.

### 3.5.1 Área gráfica não é apenas desenho

Um ponto importante é que a área gráfica não deve ser confundida com um simples editor de desenho. O que se organiza ali não são apenas formas geométricas, mas a representação do modelo que será posteriormente simulado. Em outras palavras, o espaço gráfico é também espaço semântico. Cada elemento visível ali tende a corresponder a uma entidade de modelagem relevante.

Essa observação ajuda o usuário a olhar a interface da forma correta. Em vez de pensar que está apenas manipulando objetos visuais, ele deve entender que está interagindo com a representação operacional do modelo.

## 3.6 Árvores, painéis e editores auxiliares

Além da área gráfica central, a GUI do *GenESys* oferece outros elementos igualmente importantes para o trabalho cotidiano. Esses elementos complementam a leitura visual do modelo e tornam possível inspecioná-lo de forma mais rica.

### 3.6.1 Árvores de navegação

As árvores de navegação ajudam o usuário a localizar elementos do sistema e do modelo. Dependendo do contexto, elas podem mostrar:

- *plugins*;
- componentes do modelo;
- *data definitions*;
- outras estruturas auxiliares associadas ao trabalho corrente.

Em um primeiro contato, essas árvores devem ser vistas como mecanismos de orientação. Elas ajudam a responder à pergunta: que elementos estão disponíveis ou presentes no modelo com o qual estou trabalhando?

### 3.6.2 Editor de propriedades

O editor de propriedades é um dos instrumentos mais importantes da GUI. É por meio dele que o usuário deixa de apenas observar a existência de um elemento e passa a lidar com seus atributos, parâmetros e opções de configuração. Em outras palavras, é nesse ponto que o modelo começa a se tornar efetivamente editável.

Na prática, isso significa que a seleção de um elemento na área gráfica ou em uma árvore de navegação tende a estar associada à exibição de propriedades correspondentes. Ao longo dos próximos capítulos, esse mecanismo aparecerá continuamente, porque quase toda modificação relevante de um modelo passa por algum tipo de parametrização.

### 3.6.3 Abas e visões complementares

A presença de abas centrais e de painéis organizados por abas mostra que a GUI foi construída para sustentar várias perspectivas sobre o mesmo modelo. Algumas dessas perspectivas são visuais, outras são textuais, outras são voltadas à execução, à depuração ou à apresentação de resultados. O usuário não precisa dominar tudo isso agora, mas deve reter a ideia de que o modelo não é observado apenas por uma única superfície visual.

Essa multiplicidade de visões será especialmente importante quando, mais adiante, introduzirmos a aba *Simulang* como representação textual complementar ao modelo gráfico.

## 3.7 Fluxo mínimo de orientação do usuário na GUI

Para que a familiarização com a GUI seja produtiva, convém adotar um pequeno roteiro de orientação. Em vez de explorar aleatoriamente a janela principal, o usuário pode seguir esta sequência:

1. localizar os comandos de modelo;
2. identificar a área gráfica central;
3. localizar árvores e painéis auxiliares;
4. localizar o editor de propriedades;
5. reconhecer os comandos básicos de simulação;
6. observar a existência de abas complementares.

Esse roteiro tem uma vantagem importante: ele evita que a riqueza da interface seja percebida como desorganização. Ao seguir uma ordem simples, o usuário transforma a janela principal em algo inteligível e progressivamente explorável.

## 3.8 O que não é necessário dominar agora

Um primeiro contato bem-sucedido com a GUI não exige dominar toda a aplicação. Não é necessário, por exemplo:

- usar todos os comandos de edição;
- compreender todos os painéis de depuração;
- interpretar todas as árvores da interface;
- conhecer em profundidade todos os tipos de elementos disponíveis;
- explorar ainda a totalidade das linguagens e representações do modelo.

Pelo contrário, tentar dominar tudo imediatamente tende a produzir ruído. A aprendizagem mais eficiente ocorre por camadas. Primeiro, o usuário entende a estrutura geral da interface. Depois, passa a trabalhar com modelos concretos. Em seguida, começa a relacionar a GUI com execução, resultados e linguagem textual. Esse é o percurso adotado neste manual.

## 3.9 Boas práticas no primeiro contato com a interface

Há algumas práticas simples que tornam a experiência inicial com a GUI muito mais proveitosa.

A primeira é trabalhar sempre com um objetivo concreto. Em vez de “explorar a interface”, procure localizar algo específico: o comando de abrir modelo, a área gráfica, o painel de propriedades ou os comandos de simulação.

A segunda é não confundir quantidade de recursos com dificuldade de uso. A interface do *GenESyS* é ampla porque organiza várias fases do trabalho com modelos. Isso não significa que todos os recursos precisem ser usados simultaneamente.

A terceira é observar a interface como ambiente integrado. Menus, área gráfica, árvores, propriedades, simulação e relatórios não são partes independentes. Eles formam um único fluxo de trabalho.

A quarta é aceitar que algumas partes da GUI só passarão a fazer sentido quando um modelo estiver efetivamente carregado. É por isso que o próximo capítulo entrará diretamente nos modelos de exemplo.

## 3.10 Considerações Finais

Este capítulo apresentou a interface gráfica do *GenESyS* como o principal ambiente de trabalho do usuário. Em vez de tratar a GUI como conjunto disperso de comandos, o texto procurou mostrar sua lógica de organização: a janela principal reúne recursos para gerir modelos, editá-los, parametrizá-los, simulá-los e observá-los sob diferentes perspectivas. Essa compreensão é essencial porque evita que o usuário reduza a aplicação a um simples editor gráfico ou, no extremo oposto, a perceba como uma interface excessivamente complexa e desordenada.

Com essa orientação estabelecida, o próximo passo natural é começar a trabalhar sobre modelos concretos. É nesse momento que a interface gráfica realmente ganha sentido operacional. Por isso, o capítulo seguinte passará a tratar dos modelos de exemplo já disponíveis no projeto e de seu uso inicial na GUI, permitindo ao leitor abrir, inspecionar, executar e modificar casos reais de simulação.

Teste seu entendimento desse conteúdo respondendo as seguintes perguntas:

1. Por que a GUI do *GenESyS* deve ser entendida como ambiente integrado de trabalho, e não apenas como editor gráfico?
2. Quais são os grupos funcionais mais importantes da interface em um primeiro contato?
3. Qual é a função da área gráfica do modelo dentro da experiência de uso do simulador?
4. Por que não é necessário dominar imediatamente todos os menus, árvores e painéis da aplicação?

RASCUNHO



## 4. Modelos de exemplo e uso inicial do simulador

### 4.1 Introdução

Uma vez que a interface gráfica do *GenESyS* esteja em funcionamento e que o leitor já tenha uma visão inicial de sua organização, a próxima etapa natural é começar a trabalhar com modelos concretos. Este é o ponto em que o simulador deixa de ser apenas uma aplicação aberta na tela e passa a se tornar um ambiente de modelagem e experimentação efetivo. Para isso, o caminho mais adequado não é começar criando imediatamente um modelo completo do zero, mas utilizar os modelos de exemplo já disponíveis no projeto.

Essa decisão é importante tanto do ponto de vista didático quanto do ponto de vista prático. Do ponto de vista didático, um modelo de exemplo já oferece uma estrutura coerente, com elementos interligados, propriedades preenchidas e um comportamento esperado. Isso reduz a carga cognitiva inicial do usuário, que não precisa aprender simultaneamente a interface, a semântica dos componentes, a lógica de construção do fluxo e o mecanismo de execução da simulação. Do ponto de vista prático, os modelos de exemplo permitem verificar desde cedo se a instalação do sistema está funcional e se a GUI consegue abrir, exibir, interpretar e executar modelos reais.

Neste capítulo, o objetivo é ensinar o leitor a usar os modelos salvos na pasta `models/`

como porta de entrada para o simulador. Ao final da leitura, o usuário deverá saber como localizar um modelo de exemplo, carregá-lo na interface gráfica, ler sua estrutura visual, identificar o fluxo principal do sistema modelado, executar a simulação e realizar pequenas modificações controladas para observar o efeito dessas mudanças.

## 4.2 Por que começar por modelos de exemplo

Simuladores ricos costumam oferecer muitas possibilidades de trabalho já no primeiro contato. Essa riqueza, embora valiosa, pode se transformar em obstáculo quando não se escolhe uma boa estratégia de entrada. No caso do *GenESyS*, o uso de modelos de exemplo resolve exatamente esse problema.

Ao abrir um modelo já salvo, o leitor tem acesso imediato a um artefato que:

- já está organizado de forma coerente;
- já contém uma lógica de fluxo ou estrutura de modelagem;
- já pode ser inspecionado pela GUI;
- já pode ser executado;
- já pode ser usado como base para alterações controladas.

Isso permite que o aprendizado ocorra em camadas. Primeiro, o usuário aprende a reconhecer o modelo como objeto visual dentro da interface. Depois, aprende a executá-lo. Em seguida, começa a relacionar os elementos gráficos aos seus efeitos na simulação. Só mais tarde passa a produzir modelos próprios com maior autonomia.

Em termos pedagógicos, o modelo de exemplo cumpre um papel semelhante ao de um experimento já montado em um laboratório: ele não substitui a aprendizagem ativa, mas cria um contexto inicial estável no qual o usuário pode começar a observar, formular hipóteses, modificar parâmetros e interpretar resultados.

### 4.2.1 O erro de começar do zero cedo demais

Há uma tentação natural, especialmente em leitores tecnicamente curiosos, de tentar construir um modelo próprio logo nas primeiras interações com o sistema. Embora essa iniciativa possa parecer produtiva, ela frequentemente introduz dificuldades desnecessárias.

Quando o usuário começa cedo demais a partir de um espaço vazio, ele precisa decidir ao mesmo tempo:

- que elementos inserir;
- em que ordem conectá-los;
- quais propriedades ajustar;
- quais dados de apoio são necessários;
- como verificar se o modelo está coerente;
- o que esperar da execução.

Esse acúmulo de decisões torna mais difícil distinguir o que é dificuldade própria da modelagem e o que é apenas desconhecimento inicial da ferramenta. O modelo de exemplo evita esse ruído. Ele separa o problema do uso da interface do problema da autoria completa do modelo.

#### 4.2.2 Modelos de exemplo como material de leitura

Além de servirem como casos executáveis, os modelos de exemplo também devem ser lidos. Essa leitura, porém, não é igual à leitura de um texto ou de um código-fonte. Ela envolve:

- observar o arranjo visual do modelo;
- perceber a sequência principal do fluxo;
- identificar pontos de entrada e saída;
- reconhecer elementos auxiliares que sustentam a execução;
- relacionar a estrutura visual ao comportamento observado.

Com isso, o modelo deixa de ser um arquivo e passa a ser um objeto de estudo dentro da GUI.

### 4.3 A pasta `models/` como referência de uso

Para o usuário do *GenESyS*, a pasta `models/` deve ser entendida como uma das áreas mais importantes do projeto. É nela que se encontram os modelos salvos que serão usados como ponto de entrada para a exploração do simulador. Nesta primeira parte do livro, essa pasta tem prioridade muito maior do que qualquer diretório de código-fonte.

O papel dessa pasta é duplo. Primeiro, ela concentra material concreto de uso do simulador. Segundo, ela oferece ao usuário um repertório de exemplos com os quais aprender a abrir, ler, executar e alterar modelos. Isso significa que, para fins de aprendizagem inicial, o usuário não precisa se preocupar com detalhes de implementação do software. Seu universo principal de trabalho está justamente nesses arquivos de modelo.

#### 4.3.1 O que procurar na pasta `models/`

Ao navegar pela pasta `models/`, o usuário deve procurar modelos que satisfaçam, preferencialmente, três critérios:

1. sejam visualmente simples ou moderadamente simples;
2. tenham um fluxo principal facilmente identificável;
3. permitam uma execução cujo comportamento possa ser observado sem grande esforço interpretativo.

Isso não significa que modelos maiores ou mais sofisticados não sejam valiosos. Significa apenas que, para a entrada no simulador, convém priorizar casos que favoreçam clareza e legibilidade.

### 4.3.2 Como selecionar bons primeiros exemplos

Em uma primeira aproximação, os melhores exemplos tendem a ser aqueles em que o usuário consegue responder relativamente rápido às seguintes perguntas:

- onde o fluxo começa?
- por quais elementos principais ele passa?
- onde ele termina?
- há filas, recursos, atributos ou dados auxiliares que sustentam esse comportamento?
- o que eu espero observar quando esse modelo for executado?

Se essas perguntas puderem ser respondidas sem grande dificuldade, o modelo provavelmente é um bom candidato para exploração inicial.

## 4.4 Abrindo um modelo de exemplo na GUI

Com a aplicação já iniciada, o próximo passo é abrir um modelo real. A operação de abertura de um arquivo de modelo não deve ser tratada como simples detalhe técnico, porque ela inaugura a fase em que a GUI passa a operar efetivamente sobre um objeto concreto de trabalho.

O procedimento geral é o seguinte:

1. iniciar a GUI do *GenESyS*;
2. localizar o menu ou comando correspondente à abertura de modelo;
3. navegar até a pasta `models/`;
4. selecionar um arquivo de modelo apropriado;
5. confirmar a abertura e observar a atualização da interface.

Depois disso, a área gráfica central deverá passar a exibir o modelo, e os painéis da GUI tenderão a refletir os elementos carregados, permitindo inspeção mais detalhada.

### 4.4.1 O que confirmar logo após a abertura

Assim que o modelo for carregado, o usuário deve confirmar alguns pontos simples, mas importantes:

- a área gráfica foi preenchida com a estrutura do modelo;
- há elementos e conexões visíveis;
- a interface parece reconhecer o modelo como aberto;
- comandos de inspeção e simulação se tornaram utilizáveis;
- o usuário consegue navegar visualmente pelo conteúdo carregado.

Essa confirmação é útil porque distingue dois estados muito diferentes da aplicação:

- a GUI vazia ou sem modelo ativo;
- a GUI trabalhando efetivamente sobre um modelo.

Somente no segundo caso faz sentido avançar para leitura e execução.

#### 4.4.2 Quando o modelo parece “grande demais”

Em alguns casos, o primeiro modelo aberto pode parecer visualmente denso demais. Se isso ocorrer, o usuário não deve interpretar imediatamente essa sensação como problema do simulador ou do modelo. Em geral, isso apenas indica que:

- o zoom precisa ser ajustado;
- o modelo escolhido não era o mais adequado para a primeira exploração;
- a leitura deve começar por uma região principal, e não pela totalidade da cena.

Nessa situação, a melhor prática é voltar a um exemplo mais simples ou restringir a análise ao fluxo principal mais facilmente identificável.

### 4.5 Como ler um modelo na área gráfica

Uma vez carregado o modelo, a próxima tarefa do usuário é lê-lo. Essa leitura não é puramente visual, nem puramente técnica. Ela é uma leitura orientada por perguntas sobre comportamento e estrutura.

A melhor ordem de leitura inicial é:

1. localizar o início do fluxo;
2. seguir o percurso principal do modelo;
3. identificar o término do fluxo;
4. observar elementos de apoio;
5. só depois examinar propriedades e detalhes locais.

Essa ordem é importante porque impede que o usuário se perca em detalhes antes de compreender a lógica central do modelo.

#### 4.5.1 Identificando o fluxo principal

O fluxo principal é a estrutura mais importante do modelo para o primeiro contato. Ele corresponde ao caminho essencial percorrido pelas entidades, eventos ou transformações principais representadas no sistema. Em muitos casos, esse fluxo pode ser reconhecido por uma sequência de componentes conectados de forma relativamente linear ou por um arranjo visual que sugira claramente a progressão do processo modelado.

A pergunta que deve orientar o usuário é:

Qual é a história mínima que este modelo está contando?

Quando essa pergunta começa a ser respondida, o modelo deixa de ser um conjunto de elementos na tela e passa a ser percebido como sistema organizado.

### 4.5.2 Distinguindo fluxo principal de elementos auxiliares

Uma leitura madura do modelo depende de distinguir o que constitui seu percurso central daquilo que lhe dá suporte. Muitas vezes, além do fluxo principal, o modelo também envolve dados e estruturas auxiliares: filas, recursos, estatísticas, atributos, variáveis, contadores ou outras definições.

O usuário não deve ignorar esses elementos, mas também não deve começar por eles. O procedimento correto é:

1. entender primeiro o fluxo;
2. depois perguntar que elementos de apoio permitem que esse fluxo funcione;
3. só então observar propriedades específicas.

Essa ordem melhora bastante a legibilidade do modelo.

## 4.6 Executando o modelo pela primeira vez

Depois de abrir e ler o modelo, chega o momento decisivo: executar a simulação. É nessa etapa que o usuário verifica se a estrutura visual observada realmente corresponde a um comportamento dinâmico compreensível.

Em um primeiro uso, a execução deve ser conduzida com intenção exploratória e não com pressa. O objetivo não é apenas “rodar” o modelo, mas perceber:

- se a simulação inicia corretamente;
- se o comportamento do modelo faz sentido em relação à sua estrutura;
- se a GUI oferece sinais suficientes para acompanhar o andamento;
- se o modelo chega a um estado final ou evolui de forma coerente.

### 4.6.1 O que observar durante a execução

Ao executar um modelo pela primeira vez, o usuário deve evitar tentar ler todas as saídas e todos os painéis ao mesmo tempo. É melhor concentrar-se em alguns aspectos centrais:

- há indicação clara de que a simulação começou;
- o modelo parece evoluir visualmente ou produzir sinais coerentes de progresso;
- o comportamento observado é compatível com o fluxo principal previamente identificado;
- não há falha estrutural evidente no processo de execução.

Em outras palavras, a primeira execução serve para conectar o modelo estático à sua dinâmica.

### 4.6.2 Primeira validação do comportamento

Uma validação inicial bem-feita é modesta, mas suficiente. O usuário deve ser capaz de responder:

- o modelo abriu corretamente?
- a simulação pôde ser iniciada?
- o comportamento observado parece plausível?
- a execução terminou ou evoluiu sem inconsistências evidentes?

Se essas respostas forem positivas, então o primeiro contato prático com o simulador já foi bem-sucedido.

## 4.7 Modificando um modelo de exemplo de forma controlada

Depois de abrir e executar um modelo, o passo seguinte mais produtivo é fazer pequenas modificações controladas. Essa etapa é pedagogicamente muito importante, porque transforma o modelo de exemplo em instrumento ativo de aprendizagem.

As melhores modificações iniciais são aquelas que:

- não alteram completamente a estrutura do modelo;
- podem ser facilmente revertidas;
- produzem efeito observável;
- ajudam o usuário a entender a função de propriedades ou parâmetros.

Exemplos de mudanças adequadas nesse estágio incluem:

- ajustar uma propriedade simples;
- alterar um nome ou rótulo;
- mudar um valor numérico associado a um componente;
- salvar uma cópia modificada do modelo.

### 4.7.1 Por que pequenas mudanças são melhores

Se a primeira modificação for grande demais, o usuário perde a capacidade de relacionar causa e efeito. Fica difícil saber se a mudança de comportamento observada decorre de um parâmetro específico, de uma alteração estrutural extensa ou de uma inconsistência introduzida acidentalmente.

Mudanças pequenas, ao contrário, permitem que o usuário experimente a lógica do simulador quase como em um laboratório:

1. observar o estado original;
2. alterar um ponto específico;
3. reexecutar;
4. comparar o efeito.

Essa metodologia é muito mais útil do que fazer modificações extensas logo no início.

## 4.8 Boas práticas para explorar modelos de exemplo

Algumas práticas simples tornam o uso inicial dos modelos de exemplo muito mais proveitoso.

### 4.8.1 Comece por casos menores

Modelos menores geralmente permitem leitura mais clara da interface, do fluxo e do comportamento. Eles também facilitam a associação entre estrutura visual e execução observada.

### 4.8.2 Salve cópias separadas

Ao modificar um modelo de exemplo, é recomendável salvar uma nova cópia em vez de sobrescrever imediatamente o arquivo original. Isso preserva o exemplo de referência e facilita comparações posteriores.

### 4.8.3 Use a execução como instrumento de leitura

Muitas vezes, o modelo só se torna plenamente inteligível quando é executado. Portanto, a execução não deve ser vista apenas como etapa final, mas como parte do próprio processo de compreensão do exemplo.

### 4.8.4 Volte à estrutura após executar

Uma prática muito útil é reler o modelo visualmente depois da primeira execução. O comportamento observado costuma lançar nova luz sobre a função de seus elementos, conexões e dados de apoio.

Tabela 4.1: Roteiro recomendado para explorar modelos de exemplo no GenESyS.

| Etapa                         | Objetivo                                              |
|-------------------------------|-------------------------------------------------------|
| Abrir o modelo                | Carregar um caso funcional na GUI                     |
| Ler o fluxo principal         | Entender a lógica central do sistema modelado         |
| Reconhecer elementos de apoio | Perceber que dados e estruturas sustentam o fluxo     |
| Executar a simulação          | Relacionar estrutura visual a comportamento observado |
| Alterar algo simples          | Transformar o exemplo em objeto ativo de aprendizagem |
| Reexecutar e comparar         | Observar o efeito da modificação realizada            |

## 4.9 Considerações Finais

Neste capítulo, os modelos de exemplo foram apresentados como o principal ponto de entrada do usuário no trabalho prático com o *GenESyS*. Em vez de começar pela criação integral de um novo modelo, o leitor foi orientado a abrir casos já salvos, lê-los visualmente, identificar seus fluxos principais, executá-los e realizar pequenas alterações controladas. Essa estratégia permite que a aprendizagem aconteça de forma progressiva e concreta, sem exigir do usuário, logo no início, domínio pleno de toda a ferramenta.

O ponto essencial a reter é que um modelo de exemplo não deve ser tratado como simples demonstração passiva. Ele é, ao mesmo tempo, objeto de leitura, objeto de execução e objeto de experimentação. A partir daqui, o usuário já não está apenas familiarizado com a GUI como estrutura, mas começa a usá-la efetivamente sobre modelos reais. O próximo passo natural é aprofundar a relação entre interface gráfica, execução, observação do comportamento e representação textual do modelo, o que será feito no capítulo seguinte ao introduzir a aba *Simulang*, os mecanismos de observação da execução e os primeiros procedimentos de análise do comportamento do sistema.

Teste seu entendimento desse conteúdo respondendo as seguintes perguntas:

1. Por que os modelos de exemplo são uma porta de entrada mais adequada do que a criação imediata de um modelo completo do zero?
2. Qual é a ordem mais produtiva para ler um modelo carregado na GUI?
3. O que o usuário deve validar na primeira execução de um modelo de exemplo?
4. Por que é importante modificar inicialmente apenas pequenos aspectos do modelo?

RASCUNHO



## 5. GUI, Simulacão, execução e observação do modelo

### 5.1 Introdução

Nos capítulos anteriores, o leitor colocou o *GenESyS* em funcionamento, conheceu a lógica geral da interface gráfica e começou a trabalhar com modelos de exemplo já salvos no projeto. A partir deste ponto, torna-se possível dar um passo importante: sair da leitura apenas estrutural dos modelos e passar a observá-los também em execução, acompanhando sua dinâmica e relacionando a representação gráfica à representação textual do sistema.

Este capítulo tem exatamente esse propósito. Ele aprofunda o uso da GUI em três direções complementares. Primeiro, mostra como a execução do modelo deve ser acompanhada pelo usuário, não como uma operação opaca, mas como um processo observável. Segundo, introduz a aba *Simulacão* como forma textual de representação do mesmo modelo que já foi visto na interface gráfica. Terceiro, orienta a observação dos primeiros resultados, traços e sinais de comportamento do sistema, de modo que o usuário comece a construir uma leitura mais madura da simulação.

A importância dessa etapa é grande. Um simulador realmente útil não é apenas capaz de abrir modelos e executá-los, mas de permitir que o usuário compreenda o que está acontecendo. A GUI do *GenESyS* foi pensada para oferecer diferentes perspectivas do mesmo trabalho: uma

perspectiva gráfica, uma perspectiva textual e uma perspectiva dinâmica, ligada à execução. A partir daqui, o leitor começará a articular essas três camadas.

## 5.2 Da estrutura estática à dinâmica do modelo

Quando o usuário abre um modelo e observa sua estrutura gráfica, ele vê uma descrição estática do sistema. Os componentes, conexões, dados de apoio e elementos gráficos indicam como a simulação foi organizada, mas ainda não mostram, por si sós, o comportamento temporal efetivo do modelo. A execução da simulação transforma essa estrutura estática em processo dinâmico.

Esse ponto é central para a compreensão do simulador. Um modelo não é apenas um diagrama bem organizado; ele é uma máquina de comportamento. A função do simulador é precisamente ativar essa máquina, conduzindo o avanço do tempo simulado, o processamento de eventos, a circulação de entidades e a atualização das informações relevantes do sistema.

Para o usuário, a principal consequência é esta: a execução não deve ser vista apenas como apertar o botão de iniciar. Ela deve ser entendida como a passagem da descrição do modelo para sua dinâmica operacional.

### 5.2.1 O que muda quando o modelo começa a executar

Quando a simulação é iniciada, o usuário passa a trabalhar com um novo tipo de leitura. Antes, a pergunta principal era:

Como este modelo está organizado?

Durante a execução, essa pergunta muda para:

Como este modelo se comporta ao longo do tempo?

Isso significa observar:

- se entidades estão sendo criadas, transformadas ou removidas;
- se o fluxo realmente percorre os elementos que o modelo sugere;
- se recursos, filas ou dados de apoio parecem ser utilizados de modo coerente;
- se o comportamento global da simulação faz sentido em relação à estrutura previamente lida.

## 5.3 Comandos de execução e sua lógica de uso

A GUI do *GenESyS* disponibiliza um conjunto de comandos diretamente ligados à simulação. Entre eles, destacam-se os comandos de iniciar, pausar, retomar, avançar passo a passo, parar e configurar. Para o usuário, esses comandos não devem ser percebidos apenas como botões operacionais, mas como maneiras diferentes de se relacionar com a dinâmica do modelo.

### 5.3.1 Iniciar a simulação

O comando de início da simulação é o ponto de transição entre modelo estático e comportamento observado. Antes de acioná-lo, o usuário deve garantir que:

- o modelo correto está carregado;
- a estrutura visual faz sentido;
- o sistema aparenta estar em condição de executar;
- o objetivo da observação está minimamente claro.

Essa última observação é importante. É melhor executar a simulação com alguma hipótese de leitura por exemplo, observar um fluxo específico ou o efeito de certa propriedade do que simplesmente iniciar a execução sem saber o que se pretende acompanhar.

### 5.3.2 Pausar, retomar e avançar passo a passo

Uma das grandes vantagens de uma GUI rica é permitir diferentes ritmos de observação da simulação. Em algumas situações, o usuário quer apenas verificar o comportamento global. Em outras, deseja observar trechos específicos do fluxo com mais atenção. É nesse contexto que comandos como pausar, retomar e executar passo a passo se tornam particularmente valiosos.

Esses comandos transformam a simulação em objeto de inspeção. Em vez de apenas deixar o sistema correr até o fim, o usuário pode:

- interromper temporariamente a execução;
- analisar o estado atual;
- retomar a simulação;
- ou avançar de forma controlada.

Esse modo de uso é especialmente útil em modelos de aprendizagem, em depuração inicial e na leitura do comportamento de elementos cuja função ainda não esteja totalmente clara para o usuário.

### 5.3.3 Parar e configurar

Os comandos de parada e configuração também merecem atenção. Parar a simulação não é apenas encerrar um processo, mas interromper a evolução do modelo de forma controlada. Já configurar a simulação significa ajustar condições de execução antes do início do experimento.

Embora o aprofundamento desses parâmetros dependa do modelo e do contexto, o usuário deve reter desde já que a execução não é completamente rígida: ela pode ser preparada, iniciada, observada e interrompida segundo necessidades diferentes.

## 5.4 Observando o comportamento do modelo durante a execução

A observação da execução deve ser conduzida de forma orientada. Se o usuário tentar acompanhar simultaneamente todos os sinais da GUI, toda a leitura se tornará difusa. O ideal é observar o modelo em níveis.

### 5.4.1 Primeiro nível: o fluxo global

No primeiro nível, o usuário deve observar o comportamento do modelo de forma global. As perguntas mais importantes são:

- a simulação iniciou corretamente?
- o fluxo principal parece coerente com a estrutura do modelo?
- o sistema evolui de forma plausível?
- o término da execução faz sentido?

Essa é uma leitura macroscópica. Ela não busca detalhes finos, mas consistência geral.

### 5.4.2 Segundo nível: elementos específicos

Depois de observar o comportamento global, o usuário pode concentrar-se em elementos específicos do modelo. Isso pode incluir:

- um componente cuja função ainda não esteja clara;
- um ponto do fluxo onde o comportamento parece particularmente importante;
- uma fila, recurso ou dado de apoio relevante;
- uma propriedade cujo efeito foi recentemente modificado.

Essa leitura localizada é mais produtiva depois que a dinâmica geral já foi compreendida minimamente.

### 5.4.3 Terceiro nível: comparação entre execuções

Uma prática especialmente útil é comparar a execução do modelo antes e depois de pequenas alterações. Esse procedimento ajuda o usuário a compreender não apenas como o sistema funciona, mas também como reagirá a mudanças. A GUI, nesse sentido, não é apenas interface de observação, mas também laboratório de comparação.

## 5.5 A aba *Simulang* como representação textual do modelo

Depois de o leitor já ter visto o modelo na GUI e já tê-lo observado em execução, a representação textual passa a fazer muito mais sentido. É exatamente por isso que a aba *Simulang* aparece agora, e não antes.

A função da aba *Simulang* não é substituir a GUI, mas oferecer uma segunda forma de leitura do mesmo modelo. Nessa forma de leitura, o usuário deixa de ver apenas a organização gráfica do sistema e passa a enxergar uma descrição textual correspondente. Isso é valioso por várias razões:

- torna o modelo mais explícito em termos formais;
- permite comparar a representação visual com a representação textual;
- ajuda a consolidar a compreensão do modelo;
- prepara o caminho para capítulos posteriores sobre linguagem e desenvolvimento.

### 5.5.1 Como ler a aba *Simulang*

A leitura da aba *Simulang* deve ser feita com a mesma cautela didática usada no restante da primeira parte do livro. O usuário não precisa, neste momento, dominar toda a sintaxe ou toda a semântica da linguagem. O que ele precisa é perceber que:

- a descrição textual corresponde ao modelo que já viu na GUI;
- elementos visuais do modelo possuem uma expressão textual;
- a linguagem oferece uma visão mais formal do sistema modelado.

A forma mais produtiva de iniciar essa leitura é comparar trechos da representação textual com elementos já conhecidos da interface gráfica. Em vez de tentar decifrar o texto inteiro de uma vez, o usuário deve perguntar:

Que parte do modelo gráfico esta região do texto parece descrever?

### 5.5.2 Por que a representação textual é útil ao usuário

Mesmo que o usuário continue trabalhando prioritariamente pela GUI, a representação textual lhe oferece benefícios concretos. Ela ajuda a:

- consolidar a compreensão do modelo;
- tornar mais explícita a estrutura do sistema;
- perceber o modelo além da sua disposição visual;
- começar a pensar na relação entre interface e linguagem.

Essa experiência é particularmente valiosa porque mostra, desde cedo, que o *GenESyS* não trata o modelo apenas como desenho, mas também como descrição formal.

## 5.6 Tracing, breakpoints e observabilidade inicial

Uma característica importante de um simulador bem estruturado é oferecer mecanismos de observação do comportamento do sistema em execução. No *GenESyS*, isso se expressa por diferentes meios disponíveis na GUI, incluindo traços, eventos, relatórios e mecanismos de parada ou acompanhamento controlado.

Nesta etapa do manual, o leitor não precisa ainda dominar profundamente esses mecanismos. Mas precisa entender sua função geral: eles tornam a simulação mais observável.

### **5.6.1 Tracing como leitura do comportamento**

O tracing deve ser visto como uma forma de registro do que ocorre na simulação. Ele ajuda o usuário a acompanhar o modelo não apenas pela percepção visual da interface, mas também por uma narrativa operacional do que está acontecendo.

Isso é especialmente útil quando o comportamento do modelo não é imediatamente evidente apenas pela área gráfica. Em tais casos, o tracing ajuda a responder:

- que eventos estão ocorrendo;
- em que ordem;
- com que efeitos aparentes sobre o estado do sistema.

### **5.6.2 Breakpoints como ferramenta de observação controlada**

Os breakpoints, quando utilizados, permitem interromper a simulação em pontos de interesse. Para o usuário, isso transforma a execução em objeto examinável. Em vez de deixar o modelo evoluir completamente sem intervenção, pode-se parar em estados relevantes e observar melhor o que está acontecendo.

Essa ferramenta é particularmente útil quando:

- se deseja estudar o comportamento de um trecho específico do modelo;
- se quer validar o efeito de uma modificação;
- se pretende compreender melhor a dinâmica de um elemento específico.

## **5.7 Primeira leitura dos resultados**

Além de observar a execução em andamento, o usuário precisa começar a desenvolver o hábito de interpretar os resultados produzidos pelo simulador. Aqui, novamente, a orientação deve ser progressiva.

A primeira leitura de resultados não deve ser estatisticamente sofisticada nem excessivamente ambiciosa. Ela deve responder a perguntas simples:

- a simulação se comportou como esperado?
- houve sinais coerentes de progressão e término?
- mudanças simples realizadas no modelo produziram efeitos observáveis?
- os relatórios ou sinais apresentados pela GUI parecem compatíveis com a estrutura do modelo?

Só depois dessa etapa inicial é que se torna útil avançar para leituras mais técnicas.

### 5.7.1 Resultados como continuação da leitura do modelo

Um ponto importante é que os resultados não devem ser tratados como algo separado do modelo. Eles são continuação da leitura do modelo por outros meios. O modelo gráfico mostra estrutura, a execução mostra dinâmica, e os resultados mostram consequências observáveis dessa dinâmica. Essas três dimensões devem ser sempre articuladas.

## 5.8 Boas práticas para observar modelos em execução

Algumas práticas tornam a observação da simulação muito mais proveitosa.

### 5.8.1 Defina um foco de observação

Antes de iniciar a execução, procure decidir o que observar primeiro: o fluxo global, um componente específico, um efeito de certa alteração, ou a relação entre GUI e *Simulang*. Isso evita dispersão.

### 5.8.2 Alterne entre visões

A força da GUI do *GenESyS* está em oferecer múltiplas perspectivas sobre o mesmo modelo. Alterne entre:

- estrutura gráfica;
- execução observada;
- traços ou eventos;
- representação textual em *Simulang*.

Essa alternância costuma produzir compreensão mais profunda do que qualquer visão isolada.

### 5.8.3 Faça pequenas modificações e compare

Sempre que possível, faça pequenas alterações controladas no modelo e compare o comportamento antes e depois. Essa estratégia transforma a observação em experimento guiado.

### 5.8.4 Não tente esgotar tudo em uma única execução

Uma simulação raramente revela tudo o que o modelo pode ensinar em uma única execução. É melhor fazer leituras sucessivas, cada uma com foco mais claro, do que tentar absorver tudo ao mesmo tempo.

Tabela 5.1: Estratégia recomendada para observar modelos em execução no GenESyS.

| <b>Etapas</b>                | <b>Objetivo</b>                                     |
|------------------------------|-----------------------------------------------------|
| Observar a estrutura gráfica | Compreender a organização estática do modelo        |
| Iniciar a simulação          | Transformar a estrutura em comportamento dinâmico   |
| Acompanhar o fluxo global    | Verificar consistência geral da execução            |
| Explorar tracing e eventos   | Tornar o comportamento mais observável              |
| Ler a aba <i>Simulang</i>    | Relacionar o modelo gráfico à sua descrição textual |
| Comparar execuções           | Entender o efeito de pequenas alterações no modelo  |

## 5.9 Considerações Finais

Este capítulo aprofundou o uso da GUI do *GenESyS* em uma direção essencial: a articulação entre estrutura visual do modelo, execução da simulação, observabilidade do comportamento e representação textual em *Simulang*. A principal ideia que o leitor deve reter é que essas dimensões não competem entre si. Elas se complementam. O modelo gráfico permite ver a organização do sistema, a simulação permite observar sua dinâmica, os mecanismos de tracing e pausa tornam esse comportamento mais legível, e a aba *Simulang* oferece uma forma textual de consolidar essa compreensão.

Com isso, a primeira parte do manual começa a ganhar densidade real de uso. O leitor já não apenas abriu a GUI e carregou um modelo; ele passou a observar o simulador em funcionamento e a relacionar suas várias representações. O próximo passo natural, a partir daqui, é consolidar o uso do sistema com exemplos mais orientados e preparar a transição para uma compreensão mais madura da relação entre modelagem, simulação e estrutura interna do GenESyS.

Teste seu entendimento desse conteúdo respondendo as seguintes perguntas:

1. Por que a execução de um modelo deve ser entendida como passagem da estrutura estática para a dinâmica do sistema?
2. Qual é a utilidade de comandos como pausar, retomar e avançar passo a passo durante a simulação?
3. Por que a aba *Simulang* só deve ser introduzida depois que o usuário já viu e executou o modelo na GUI?
4. Como tracing, breakpoints e resultados complementam a observação visual da execução do modelo?



## 6. Organização do código-fonte e arquitetura geral

### 6.1 Introdução

A partir deste ponto, o livro entra explicitamente em sua segunda metade: o manual do desenvolvedor. A mudança de perspectiva é importante. Até aqui, o foco esteve no uso do *GenESyS* como ferramenta de modelagem e simulação, com ênfase na GUI, nos modelos de exemplo e na observação do comportamento do sistema. Agora, o foco passa a ser a compreensão do próprio software: sua organização interna, seus subsistemas centrais, a forma como o código está distribuído e os pontos a partir dos quais um desenvolvedor pode começar a estudá-lo, modificá-lo e ampliá-lo.

Antes de estudar o núcleo do simulador ou discutir *plugins*, parser e aplicações, é necessário construir uma visão de conjunto da organização do repositório. Sem essa visão, o desenvolvedor tende a se perder em arquivos e classes sem perceber a função arquitetural de cada região do código. A meta deste capítulo é justamente evitar esse problema. Em vez de apresentar imediatamente detalhes locais, ele organiza o mapa geral do projeto.

A ideia central é simples. O *GenESyS* não deve ser lido como um conjunto amorfo de arquivos C++, mas como um sistema dividido em áreas com responsabilidades distintas. Algumas dessas áreas sustentam o núcleo da simulação. Outras organizam o mecanismo de

extensibilidade. Outras ainda implementam aplicações, ferramentas e testes. Compreender essa divisão é o primeiro passo para qualquer leitura técnica madura do projeto.

## 6.2 Do simulador como aplicação ao simulador como software

Na primeira metade do livro, o *GenESyS* apareceu ao leitor principalmente como aplicação. O usuário abriu a GUI, carregou modelos, executou simulações e observou resultados. Para o desenvolvedor, porém, essa visão é apenas a superfície de um sistema mais amplo. A aplicação gráfica não esgota o projeto; ela é uma de suas formas de apresentação.

Essa observação é importante porque ajuda a reposicionar corretamente o objeto de estudo. O desenvolvedor não trabalha apenas sobre uma interface gráfica, nem apenas sobre um conjunto de modelos. Ele trabalha sobre um software que:

- mantém um kernel de simulação;
- representa modelos internamente;
- administra eventos, entidades e tempo simulado;
- interpreta expressões e linguagem de simulação;
- carrega e integra *plugins*;
- sustenta aplicações distintas sobre a mesma base;
- oferece infraestrutura para testes e evolução.

Portanto, o ponto de partida da segunda metade do livro não é a GUI em si, mas o sistema do qual a GUI faz parte.

## 6.3 A lógica geral da árvore de diretórios

O repositório do *GenESyS* deve ser lido como uma árvore organizada por responsabilidade técnica. Nem todos os diretórios têm a mesma importância para o desenvolvedor, e uma boa leitura inicial consiste justamente em distinguir o que é estrutural do que é apenas contextual.

Em termos gerais, há três grandes zonas de interesse:

1. a zona do código-fonte principal;
2. a zona dos modelos e artefatos de uso;
3. a zona de apoio ao desenvolvimento, build e testes.

Dentro da primeira dessas zonas, a pasta *source/* concentra o núcleo efetivo do software. É ela que passa a dominar a leitura a partir deste capítulo. A pasta *models/*, que na primeira metade do livro servia como principal porta de entrada para o usuário, continua importante, mas agora muda de papel: deixa de ser apenas material de uso e passa a ser também material de observação arquitetural, pois ajuda o desenvolvedor a relacionar a representação do modelo ao comportamento do código.

### 6.3.1 Por que a pasta `source/` é o centro da leitura

Para o desenvolvedor, a pasta `source/` é o coração do projeto. É nela que se encontram os subsistemas fundamentais do *GenESyS*:

- `kernel`;
- `parser`;
- *plugins*;
- aplicações;
- ferramentas;
- testes.

Essa divisão já sugere uma primeira leitura arquitetural do sistema. O `kernel` representa a infraestrutura central da simulação. O `parser` cuida da linguagem e das expressões. Os *plugins* materializam o mecanismo de extensibilidade. As aplicações expõem formas concretas de uso da infraestrutura. As ferramentas complementam o ecossistema. Os testes organizam a validação do comportamento do software.

### 6.3.2 Por que `models/` continua relevante ao desenvolvedor

Na primeira parte do livro, os modelos eram tratados como material de uso do simulador. Agora, eles passam também a servir como instrumentos de leitura do software. Isso ocorre porque o desenvolvedor precisa constantemente relacionar duas coisas:

- a forma como o sistema se apresenta ao usuário;
- a forma como o sistema está implementado internamente.

Os modelos em `models/` ajudam exatamente nessa ponte. Eles mostram o tipo de artefato que a aplicação manipula e ajudam a relacionar a estrutura dos modelos ao conjunto de classes e mecanismos do `kernel`.

## 6.4 As grandes áreas de `source/`

A leitura de `source/` deve começar por suas grandes áreas, e não por arquivos isolados. Essa abordagem reduz ruído e ajuda a construir uma visão arquitetural antes da leitura detalhada de classes.

### 6.4.1 `source/kernel`

A pasta `source/kernel` concentra a infraestrutura central do simulador. É nela que o desenvolvedor encontra os elementos mais fundamentais do sistema: representação do modelo, entidades, eventos, mecanismos de simulação, utilitários e estatísticas. Esta é, sem dúvida, a região mais importante do projeto para quem deseja compreender o *GenESyS* em profundidade.

Do ponto de vista arquitetural, a pasta `kernel` organiza o que o simulador é indepen-

dentemente de qualquer interface específica. Aplicações gráficas, aplicações de terminal, ferramentas e outros módulos só conseguem existir porque esse núcleo lhes oferece uma base estável.

#### 6.4.2 `source/parser`

A pasta `source/parser` reúne a infraestrutura associada à interpretação da linguagem e das expressões do simulador. Ela é especialmente importante porque muitos parâmetros do modelo não são apenas valores literais, mas expressões textuais que precisam ser interpretadas dentro do contexto do modelo e da simulação.

Na primeira parte do livro, essa dimensão apareceu indiretamente na aba *Simulang*. Aqui, ela passa a ser entendida como subsistema de software, responsável por conectar texto, semântica do modelo e execução.

#### 6.4.3 `source/plugins`

A pasta `source/plugins` organiza um dos aspectos mais característicos do GenESyS: sua extensibilidade. Em vez de concentrar toda a funcionalidade em um núcleo fechado, o projeto prevê a incorporação de módulos especializados que ampliam os componentes, os dados e outras capacidades do sistema.

Para o desenvolvedor, isso significa que parte importante da evolução do simulador acontece nessa fronteira entre kernel e *plugins*. Muitos dos elementos de modelagem mais visíveis para o usuário são fornecidos justamente por esse mecanismo.

#### 6.4.4 `source/applications`

A pasta `source/applications` contém as aplicações construídas sobre o núcleo do sistema. É aí que o GenESyS deixa de aparecer apenas como infraestrutura e passa a assumir formas de interação concretas, como a GUI e outras aplicações auxiliares. Essa camada é importante porque mostra uma separação arquitetural valiosa: o núcleo da simulação não se confunde com uma única aplicação.

Isso significa que o software pode ser lido em dois níveis:

- como infraestrutura;
- como conjunto de aplicações sobre essa infraestrutura.

#### 6.4.5 `source/tools`

A pasta `source/tools` reúne ferramentas que apoiam o uso, o desenvolvimento ou a evolução do sistema. Embora nem sempre sejam a primeira prioridade de leitura, essas ferramentas podem se tornar importantes em contextos de depuração, suporte, integração ou manutenção.

6.4.6 source/tests

A pasta `source/tests` representa a dimensão de validação do projeto. Ela é especialmente importante para o desenvolvedor porque oferece um caminho concreto para verificar comportamento esperado, prevenir regressões e dar segurança a mudanças no sistema.

Em projetos em evolução, a existência de uma árvore de testes organizada é um dos principais sinais de maturidade técnica. No GenESyS, isso é particularmente relevante porque o software possui múltiplas camadas e mecanismos de extensibilidade.

6.5 Uma leitura em camadas do sistema

Uma forma muito útil de compreender a arquitetura geral do *GenESyS* é imaginar o sistema em camadas.

Na camada mais profunda, está o kernel. É ele que define as estruturas e serviços fundamentais da simulação. Sobre essa camada, aparecem mecanismos de linguagem e extensibilidade, como parser e *plugins*. Mais acima, surgem aplicações concretas, como a GUI, que apresentam ao usuário uma forma de interação com essa infraestrutura. Em paralelo, testes e ferramentas ajudam a sustentar a manutenção e a evolução do projeto.

Essa leitura em camadas ajuda o desenvolvedor a evitar um erro comum: confundir o que é estrutural com o que é apenas superfície de uso. A GUI é muito importante, mas ela não é o coração do sistema. Da mesma forma, um plugin pode ser central para um domínio de modelagem específico, mas ainda assim depende do kernel para existir.

Tabela 6.1: Leitura em camadas da arquitetura geral do GenESyS.

| Camada                   | Função principal                                                                                                              |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Kernel                   | Infraestrutura central da simulação, representação do modelo, eventos, entidades, tempo, estatísticas e serviços fundamentais |
| Parser e extensibilidade | Interpretação de expressões, linguagem do simulador e ampliação funcional por <i>plugins</i>                                  |
| Applications             | Formas concretas de interação com o sistema, como a interface gráfica                                                         |
| Tools e Tests            | Apoio ao desenvolvimento, validação, manutenção e evolução do projeto                                                         |

6.6 O papel da classe Simulator na visão de conjunto

A classe `Simulator` fornece uma síntese particularmente útil dessa visão geral. Ela se apresenta como a classe de mais alto nível do kernel de simulação e expõe, por meio de acessos di-

retos, diversos gerenciadores centrais do sistema. Isso inclui, entre outros, o `PluginManager`, o `ModelManager`, o `TraceManager`, o `ParserManager` e o `ExperimentManager`.

Essa observação é valiosa porque mostra, de forma muito concreta, que a arquitetura do GenESyS não se organiza em torno de um único arquivo ou de uma única classe operacional, mas em torno de um núcleo que coordena múltiplos subsistemas especializados. Em outras palavras, o desenvolvedor deve entender o `Simulator` como ponto de acesso ao ecossistema do simulador, e não apenas como objeto técnico isolado.

### 6.6.1 Por que isso importa para a leitura do código

Se o desenvolvedor entrar no projeto apenas pela GUI ou por um componente específico de modelagem, corre o risco de ler o sistema de forma fragmentada. A presença da classe `Simulator` como agregadora de gerenciadores ajuda a recentrar a leitura: o software é mais bem compreendido quando visto como um conjunto de subsistemas coordenados, e não como coleção de partes independentes.

Essa percepção será particularmente importante no próximo capítulo, quando o foco se deslocar para o kernel e para as classes que efetivamente controlam a vida do modelo em simulação.

## 6.7 Leituras erradas que este capítulo evita

Este capítulo pretende evitar algumas leituras equivocadas bastante comuns em projetos como o GenESyS.

A primeira é imaginar que a GUI seja o sistema inteiro. Ela é fundamental para o usuário, mas é apenas uma aplicação sobre uma infraestrutura mais ampla.

A segunda é imaginar que o kernel possa ser lido sem relação com parser, *plugins* e aplicações. Embora o kernel seja a base, ele não vive isolado.

A terceira é começar o estudo do código por arquivos aleatórios. Essa prática costuma produzir familiaridade local, mas não compreensão arquitetural.

A quarta é tratar a pasta `models/` como algo irrelevante para o desenvolvedor. Na verdade, ela é importante porque ajuda a relacionar a implementação com o objeto real de trabalho do simulador.

## 6.8 Estratégia recomendada de leitura do código

A forma mais produtiva de estudar o código-fonte do *GenESyS* é progressiva.

Primeiro, construa uma visão geral da árvore do projeto e das responsabilidades de suas grandes áreas. Depois, entre no kernel e estude as classes centrais que articulam o modelo,

a simulação, os eventos e a observabilidade. Em seguida, avance para as classes-base de componentes, *data definitions*, parser e *plugins*. Só então faz sentido aprofundar aplicações específicas, ferramentas e testes.

Essa ordem não é arbitrária. Ela acompanha a própria lógica do software:

1. primeiro, a infraestrutura;
2. depois, os mecanismos de ampliação;
3. depois, as aplicações concretas;
4. por fim, os instrumentos de manutenção e validação.

## 6.9 Considerações Finais

Este capítulo teve a função de reorganizar o olhar do leitor sobre o *GenESyS*. A partir dele, o projeto deixa de ser visto apenas como aplicação gráfica e passa a ser compreendido como sistema de software distribuído em áreas com responsabilidades distintas. O ponto mais importante a reter é que a arquitetura do *GenESyS* precisa ser lida por camadas: kernel, parser, *plugins*, aplicações, ferramentas e testes. Sem essa visão de conjunto, o estudo das classes individuais tende a ficar fragmentado e pouco produtivo.

Com essa base estabelecida, o passo seguinte é entrar na região mais importante do projeto: o núcleo do simulador. É nele que se definem o modelo, a simulação, os eventos, o tempo simulado, a lista de eventos futuros e os mecanismos centrais de observação do comportamento do sistema. Por isso, o capítulo seguinte se concentrará no kernel propriamente dito, com ênfase nas classes que articulam a dinâmica da simulação.

Teste seu entendimento desse conteúdo respondendo as seguintes perguntas:

1. Por que a pasta *source/* deve ser tratada como o centro da leitura técnica do projeto?
2. Qual é a vantagem de interpretar o *GenESyS* em camadas, em vez de lê-lo como conjunto disperso de arquivos?
3. Por que a GUI não deve ser confundida com o sistema inteiro?
4. Que papel a pasta *models/* ainda desempenha para o desenvolvedor, mesmo na segunda metade do livro?

RASCUNHO



## 7. Núcleo do simulador

### 7.1 Introdução

Se o capítulo anterior organizou o mapa geral do projeto, este capítulo entra na região mais importante desse mapa: o núcleo do simulador. É aqui que o *GenESyS* deixa de ser visto apenas como aplicação e passa a ser entendido como sistema que representa modelos, agenda eventos, controla repetições, mantém o tempo simulado e coordena a observabilidade do experimento.

O estudo do kernel é decisivo por uma razão simples: tudo o que o usuário faz na GUI e tudo o que o desenvolvedor adiciona por parser, *plugins* ou aplicações depende, em última instância, da infraestrutura definida aqui. Não importa se o modelo foi aberto visualmente, descrito textualmente ou construído por código. Em todos os casos, ele precisará ser representado, verificado, executado e observado por classes centrais do kernel.

Neste capítulo, o foco recairá sobre um conjunto de classes e estruturas particularmente importantes:

- `Simulator`;
- `Model`;
- `ModelSimulation`;

- `OnEventManager`;
- lista de eventos futuros;
- entidades e eventos correntes;
- mecanismos de controle do ciclo da simulação.

O objetivo não é ainda esgotar todas as classes do kernel, mas construir uma compreensão firme da dinâmica central do GenESyS.

## 7.2 O kernel como infraestrutura de simulação

O kernel do *GenESyS* deve ser entendido como a camada que torna possível a existência do simulador independentemente da interface usada. A GUI, a aplicação terminal, modelos textuais, *plugins* e ferramentas podem variar; o kernel, porém, é a base que sustenta a representação do modelo e a execução da simulação.

Em termos arquiteturais, isso significa que o kernel precisa resolver pelo menos quatro problemas fundamentais:

1. como representar um modelo de simulação;
2. como representar o estado da execução desse modelo;
3. como avançar no tempo simulado por meio de eventos;
4. como tornar a simulação observável e controlável.

Esses quatro problemas aparecem com clareza nas classes centrais do projeto. É por isso que estudar o kernel não é estudar “mais um pacote” do sistema, mas a parte que define o comportamento essencial do simulador.

## 7.3 A classe *Simulator* como ponto de entrada do kernel

A classe *Simulator* ocupa posição especial na visão do kernel. Ela se apresenta como a classe principal do sistema de simulação e fornece acesso a gerenciadores importantes como *PluginManager*, *ModelManager*, *TraceManager*, *ParserManager* e *ExperimentManager*. Essa organização já revela algo importante: o GenESyS não foi estruturado em torno de uma única classe monolítica, mas em torno de um núcleo que coordena subsistemas especializados.

Do ponto de vista do desenvolvedor, a classe *Simulator* é útil por dois motivos. Primeiro, porque oferece uma visão sintética da arquitetura do projeto. Segundo, porque funciona como ponto de acesso de alto nível aos demais mecanismos do sistema. Em vez de entrar no software por fragmentos isolados, o leitor pode começar pela percepção de que existe uma classe central que articula o ecossistema do simulador.

### 7.3.1 O que essa classe sugere sobre a arquitetura

A mera presença dos gerenciadores acessíveis a partir de `Simulator` já indica cinco pilares arquiteturais:

- o sistema administra modelos;
- o sistema administra *plugins*;
- o sistema possui uma infraestrutura explícita de tracing;
- o sistema possui um subsistema de parser;
- o sistema prevê suporte a experimentos.

Mesmo antes de ler os detalhes de cada gerenciador, o desenvolvedor já percebe que o núcleo do GenESyS foi pensado como infraestrutura coordenadora e não como bloco único de execução.

## 7.4 A classe `Model` como centro do modelo de simulação

A classe `Model` merece atenção especial porque ela representa, no kernel, o próprio modelo de simulação. Sua importância é explicitada já na documentação do cabeçalho: ela é descrita como provavelmente a classe mais importante do kernel do GenESyS. Essa formulação faz sentido. É nela que se articulam componentes, *data definitions*, lista de eventos futuros, controles, respostas, persistência, parser, verificação de consistência, gerenciamento de entidades e acesso ao estado da simulação.

Em termos práticos, `Model` é a classe na qual o modelo deixa de ser apenas ideia ou diagrama e passa a ser objeto manipulável pelo software.

### 7.4.1 Por que `Model` é tão central

A centralidade de `Model` decorre do fato de que ela articula vários mecanismos essenciais ao mesmo tempo:

- inserção e remoção de elementos do modelo;
- criação e remoção de entidades;
- envio de entidades entre componentes;
- verificação de consistência do modelo;
- persistência e carregamento;
- acesso a parser, tracing e simulação;
- manutenção da lista de eventos futuros.

Isso significa que a classe `Model` não é apenas um contêiner de componentes. Ela é o centro organizador da vida do modelo dentro do simulador.

### 7.4.2 Modelo como combinação de componentes e dados

A própria documentação da classe indica que o modelo é representado principalmente por:

- uma coleção de componentes de modelo, adequadamente configurados e conectados;
- uma coleção de *model data definitions*, que sustentam a infraestrutura do modelo.

Essa distinção é arquiteturalmente fundamental. Ela antecipa um princípio que voltará em capítulos posteriores: o modelo não é feito apenas de blocos de fluxo. Ele depende também de uma camada de dados e definições que dá suporte à simulação.

### 7.4.3 A lista de eventos futuros

Entre os atributos de `Model`, um merece destaque especial: a lista de eventos futuros. A própria documentação do código a descreve como um dos elementos mais importantes de um sistema de simulação orientado a eventos. Isso se justifica facilmente. É a partir dessa lista que a simulação avança cronologicamente, tomando sempre o próximo evento a ser processado.

Para o desenvolvedor, isso é essencial. O tempo simulado não avança como simples laço incremental arbitrário. Ele avança por disparo ordenado de eventos agendados, e a lista de eventos futuros é o mecanismo que sustenta esse processo.

## 7.5 A classe `ModelSimulation`

Se `Model` representa o modelo, `ModelSimulation` representa a execução desse modelo. Essa distinção é extremamente importante. O modelo e sua simulação são objetos relacionados, mas não idênticos. O primeiro descreve a estrutura do sistema modelado; o segundo controla seu ciclo de execução.

A documentação de `ModelSimulation` é particularmente esclarecedora. Ela a descreve como a camada madura e funcional de execução da simulação, baseada em repetições, comprimento de replicação, configuração de *warm-up* e processamento de eventos. Também enfatiza que, embora existam abstrações mais altas de experimentação em desenvolvimento, `ModelSimulation` continua sendo o núcleo estável de execução. Essa observação é extremamente valiosa para o desenvolvedor, porque indica qual classe realmente concentra hoje o comportamento operacional maduro da simulação.

### 7.5.1 Responsabilidades de `ModelSimulation`

As responsabilidades de `ModelSimulation` podem ser organizadas em alguns grupos principais:

- iniciar, pausar, avançar passo a passo e parar a simulação;
- controlar repetições;
- manter o tempo simulado corrente;
- administrar comprimento de replicação e período de *warm-up*;
- verificar condições de término;
- gerenciar breakpoints;
- disparar relatórios;

- coordenar o processamento de eventos.

Observe-se que essa classe não é mero detalhe operacional. Ela concentra a lógica efetiva da execução do modelo e, por isso, deve ser estudada com cuidado por qualquer desenvolvedor que pretenda alterar a dinâmica central do simulador.

### 7.5.2 O ciclo da simulação

A leitura da classe `ModelSimulation` permite perceber uma estrutura lógica clara do ciclo de simulação:

1. inicialização da simulação;
2. inicialização de uma replicação;
3. processamento sequencial de eventos;
4. verificação de pontos de interrupção e condições de término;
5. encerramento de replicação;
6. encerramento da simulação.

Esse ciclo é muito importante para a compreensão do sistema. Ele mostra que a execução não é uma simples chamada monolítica de “run”, mas um processo estruturado em fases, cada uma com responsabilidades próprias.

### 7.5.3 Tempo simulado, replicações e *warm-up*

Um dos méritos da classe `ModelSimulation` é tornar explícito que a execução do modelo envolve mais do que a simples passagem de eventos. Há também a gestão de:

- tempo simulado corrente;
- número de repetições;
- comprimento de replicação;
- período de *warm-up*;
- unidade de tempo de base;
- condição de término.

Esses elementos revelam que a classe foi desenhada para sustentar simulação orientada a experimentação e análise, e não apenas execução ad hoc de um fluxo de eventos.

## 7.6 Eventos, evento corrente e processamento sequencial

A simulação discreta orientada a eventos depende de uma ideia central: o sistema evolui de evento em evento. No *GenESyS*, essa lógica aparece de forma muito clara na relação entre `Model`, `ModelSimulation`, `Event` e a lista de eventos futuros.

Durante a simulação, há sempre um evento corrente em processamento. Esse evento define o instante simulado atual e o contexto da operação em curso. Ao ser processado, ele pode produzir efeitos sobre entidades, componentes, dados do modelo e a própria agenda

futura do sistema.

Essa organização tem duas implicações importantes para o desenvolvedor:

1. o estado do sistema não é atualizado de forma difusa, mas em pontos discretos de processamento;
2. a semântica dos componentes e de grande parte das ações do modelo depende do evento corrente.

Essa observação se conecta diretamente ao parser, a entidades, a componentes e a várias expressões ligadas ao estado do modelo, que serão discutidas em capítulos posteriores.

## 7.7 A classe `OnEventManager` e a observabilidade do núcleo

Se `ModelSimulation` controla a execução, `OnEventManager` torna essa execução observável por meio de eventos de alto nível. Esse é um ponto arquitetural muito elegante do GenESyS e merece destaque especial.

A classe `OnEventManager` permite registrar tratadores para diversos tipos de ocorrência do ciclo de vida do modelo e da simulação. Entre eles estão:

- sucesso na verificação do modelo;
- carregamento e salvamento de modelo;
- início e término de replicação;
- processamento de evento;
- criação, movimentação e remoção de entidades;
- início, pausa, retomada e fim da simulação;
- disparo de breakpoint.

Essa infraestrutura mostra que o GenESyS trata observabilidade como parte constitutiva do kernel, e não como mero recurso periférico de interface.

### 7.7.1 Por que isso é arquiteturalmente importante

O `OnEventManager` separa elegantemente duas coisas:

- a lógica de execução da simulação;
- a lógica de reação a eventos relevantes da simulação.

Essa separação é valiosa porque permite que aplicações, ferramentas, relatórios e mecanismos de depuração se conectem ao comportamento do kernel sem precisar misturar sua lógica diretamente ao processamento central. Trata-se, em essência, de um mecanismo de observação e reação baseado em eventos do próprio simulador.

### 7.7.2 ModelEvent e SimulationEvent

A presença das classes auxiliares `ModelEvent` e `SimulationEvent` reforça essa arquitetura. Elas funcionam como objetos de contexto para os tratadores registrados, carregando informações sobre o modelo, o tempo simulado, o evento corrente, a replicação, o estado de pausa ou parada, a entidade criada e o componente de destino, entre outros dados relevantes.

Com isso, o `OnEventManager` não apenas anuncia que “algo ocorreu”, mas fornece contexto suficiente para que a reação a esse evento seja tecnicamente útil.

## 7.8 Breakpoints e controle fino da execução

A classe `ModelSimulation` também torna explícita a existência de listas de breakpoints por tempo, componente e entidade. Essa característica é particularmente relevante porque mostra que o kernel foi concebido não apenas para executar modelos até o fim, mas para permitir observação controlada da simulação.

Do ponto de vista do usuário, isso aparece na GUI como capacidade de pausar ou avançar passo a passo. Do ponto de vista do desenvolvedor, isso revela que o controle fino da execução não é adicionado pela interface de forma superficial. Ele está previsto na própria infraestrutura do kernel.

Essa é uma decisão arquitetural importante. Ela significa que depuração, inspeção e experimentação controlada não dependem exclusivamente da aplicação gráfica; elas têm fundamento no núcleo de simulação.

## 7.9 Uma leitura integrada do núcleo

A melhor forma de compreender o núcleo do *GenESyS* é integrar as funções das classes estudadas até aqui.

- `Simulator` coordena o acesso a subsistemas centrais.
- `Model` representa o modelo, seus componentes, dados e eventos futuros.
- `ModelSimulation` controla o ciclo de execução desse modelo.
- `OnEventManager` torna os eventos relevantes observáveis e reagíveis.

Essa integração ajuda a perceber que o kernel não é um conjunto de classes paralelas sem articulação. Ele é uma infraestrutura coerente na qual cada classe ocupa papel específico dentro da mesma dinâmica de simulação.

## 7.10 Considerações Finais

Este capítulo apresentou o núcleo do simulador a partir de suas classes mais centrais. O ponto principal a reter é que o *GenESyS* organiza sua infraestrutura de simulação de forma clara: há

Tabela 7.1: Papéis das classes centrais do núcleo do GenESyS.

| Classe                       | Papel principal                                                                                                       |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <code>Simulator</code>       | Classe de mais alto nível do kernel, coordenando acesso a gerenciadores centrais                                      |
| <code>Model</code>           | Representação do modelo de simulação, de seus componentes, dados, entidades e eventos futuros                         |
| <code>ModelSimulation</code> | Controle da execução do modelo, incluindo repetições, tempo simulado, condições de término e processamento de eventos |
| <code>OnEventManager</code>  | Registro e disparo de tratadores associados a eventos relevantes do modelo e da simulação                             |

uma classe de alto nível que coordena subsistemas, uma classe que representa o modelo, uma classe que controla a execução desse modelo e um mecanismo explícito de observabilidade orientada a eventos. Essa organização é suficientemente robusta para sustentar aplicações, parser, *plugins*, tracing, breakpoints e futuras extensões.

Com essa base estabelecida, o próximo passo natural é avançar para a estrutura dos próprios elementos do modelo. Em outras palavras, se este capítulo mostrou como o sistema representa e executa um modelo, o capítulo seguinte mostrará de que tipos de objetos esse modelo é feito: componentes, *data definitions* e mecanismos de extensibilidade por *plugins*.

Teste seu entendimento desse conteúdo respondendo as seguintes perguntas:

1. Qual é a diferença conceitual entre `Model` e `ModelSimulation` no kernel do GenESyS?
2. Por que a lista de eventos futuros é uma estrutura tão importante na simulação orientada a eventos?
3. Que papel o `OnEventManager` desempenha na observabilidade do sistema?
4. Por que a presença de breakpoints no kernel é uma decisão arquitetural relevante e não apenas um detalhe de interface?



## 8. Componentes, *model data* e sistema de *plugins*

### 8.1 Introdução

Depois de estudar o núcleo do simulador em seu nível mais central, o próximo passo é compreender de que objetos concretos o modelo é feito. Em outras palavras, se o capítulo anterior respondeu à pergunta “como o GenESyS representa e executa um modelo?”, este capítulo responde a outra pergunta igualmente importante: “quais são os elementos que compõem esse modelo dentro do software?”.

No GenESyS, essa resposta não se reduz a um único tipo de classe. O modelo é construído a partir de uma distinção arquitetural fundamental entre:

- componentes de modelo;
- *model data definitions*.

Essa distinção é especialmente importante porque organiza toda a forma como o simulador representa o fluxo principal de um sistema e a infraestrutura de dados que o sustenta. Além disso, é a partir dela que o mecanismo de extensibilidade por *plugins* ganha clareza. Novos componentes e novos dados do modelo podem ser introduzidos ao sistema sem que o núcleo precise ser reescrito como bloco monolítico.

Este capítulo aprofunda essa arquitetura. Primeiro, discute a classe-base `ModelDataDefinition`. Depois, mostra como `ModelComponent` a especializa para representar comportamento no fluxo do modelo. Em seguida, discute a importância de dados internos e anexados, a relação entre composição e agregação e, por fim, situa o mecanismo de *plugins* como forma de ampliar o sistema de maneira consistente.

## 8.2 `ModelDataDefinition` como classe-base do modelo

A classe `ModelDataDefinition` ocupa papel estrutural no GenESyS. Sua documentação a descreve como a base para qualquer dado do modelo – filas, recursos, variáveis e outros elementos – e também para qualquer componente do modelo. Isso significa que ela define uma infraestrutura comum que será compartilhada tanto por elementos que sustentam a simulação quanto por elementos que participam diretamente do fluxo do modelo.

Essa decisão de projeto é extremamente importante. Em vez de separar completamente os componentes do fluxo e os dados de apoio em hierarquias sem relação, o GenESyS faz com que ambos compartilhem uma base comum. Com isso, o software ganha:

- um mecanismo uniforme de identificação e nomeação;
- suporte comum à persistência;
- suporte comum à verificação;
- infraestrutura comum para criação de dados internos e anexados;
- mecanismos padronizados de tracing e controles de simulação.

Em termos arquiteturais, isso torna o modelo mais coerente. Em vez de dois universos sem comunicação, há uma hierarquia unificada na qual componentes e dados são espécies diferentes de elementos do modelo.

### 8.2.1 Responsabilidades centrais de `ModelDataDefinition`

A leitura da classe mostra algumas responsabilidades particularmente relevantes:

- manter identidade e nome do elemento;
- oferecer infraestrutura de persistência;
- permitir verificação local do elemento;
- sustentar dados internos e anexados;
- oferecer integração com parser e controles de simulação;
- prover mecanismos de tracing;
- definir ponto de extensão para criação de dados relacionados.

Isso significa que `ModelDataDefinition` não é apenas uma superclasse abstrata conveniente. Ela é um bloco real de infraestrutura sobre o qual o restante do modelo se apoia.

## 8.2.2 Internal data e attached data

Um dos aspectos mais interessantes da classe `ModelDataDefinition` é a distinção explícita entre *internal data* e *attached data*. Essa distinção, que já foi importante em discussões anteriores do livro, aparece aqui como parte da infraestrutura base do sistema.

- **Internal data** corresponde a elementos internos ao objeto, mantidos por composição.
- **Attached data** corresponde a dados referenciados, mantidos por agregação.

Essa distinção é arquiteturalmente muito valiosa. Ela permite que o software represente, de forma clara, tanto elementos que pertencem estruturalmente a um componente quanto elementos que apenas lhe são associados. Em outras palavras, o GenESyS não trata todas as relações entre objetos do modelo da mesma maneira, e isso melhora muito a clareza da semântica interna do sistema.

## 8.2.3 Persistência e verificação como responsabilidades da base

Outro aspecto importante é que persistência e verificação já aparecem na classe-base. Isso mostra que o sistema espera que elementos do modelo sejam:

- carregados;
- salvos;
- validados;
- reconstituídos;
- preparados para simulação.

Ao concentrar essa infraestrutura na base, o GenESyS evita repetição desnecessária nas subclasses e impõe uma disciplina consistente às extensões do modelo.

## 8.3 ModelComponent como especialização comportamental

A classe `ModelComponent` herda de `ModelDataDefinition` e acrescenta a ela uma dimensão decisiva: comportamento orientado por chegada de entidade e disparo de evento. Sua documentação a descreve como um bloco que representa um comportamento específico a ser simulado e que é ativado quando uma entidade chega ao componente.

Essa formulação é extremamente importante porque define o lugar dos componentes na arquitetura do GenESyS. Um componente não é apenas um objeto visual nem apenas um bloco nominal do modelo. Ele é uma unidade de comportamento dentro do fluxo do sistema modelado.

### 8.3.1 A ideia de fluxo entre componentes

Um modelo construído com `ModelComponent` é, em essência, um conjunto de componentes interconectados. O comportamento global do modelo emerge do percurso das entidades por

esse fluxo. Isso explica por que a classe mantém um `ConnectionManager` e oferece operações de conexão entre componentes.

Essa é uma das grandes diferenças entre `ModelComponent` e um *model data definition* genérico:

- um componente participa diretamente da lógica do fluxo;
- um dado do modelo pode sustentar esse fluxo sem ser ele próprio parte do percurso principal.

### 8.3.2 Despacho de eventos e `_onDispatchEvent`

A classe `ModelComponent` deixa claro, por sua interface, que cada componente concreto deve implementar um comportamento específico de despacho. Isso aparece no método protegido e virtual puro `_onDispatchEvent(Entity*, unsigned int)`. É nesse ponto que a semântica própria de cada componente concreto passa a existir.

Essa escolha arquitetural é particularmente boa porque separa:

- a infraestrutura comum dos componentes;
- o comportamento particular de cada tipo de componente.

Com isso, o GenESyS consegue ter uma classe-base forte, mas ainda assim manter subclasses altamente especializadas.

### 8.3.3 Componentes como *model data* especializados

O fato de `ModelComponent` herdar de `ModelDataDefinition` tem consequências profundas:

- um componente também tem nome, identidade e persistência;
- um componente também pode possuir dados internos e anexados;
- um componente participa da infraestrutura comum de verificação;
- um componente integra-se naturalmente ao restante da semântica do modelo.

Isso evita uma separação artificial entre estrutura de dados e estrutura comportamental. Em vez disso, o sistema reconhece que um componente é também um elemento do modelo, apenas dotado de papel comportamental específico.

## 8.4 Fluxo principal e dados de apoio

Uma das ideias mais importantes que o desenvolvedor precisa consolidar é a diferença entre:

- o fluxo principal do modelo;
- os dados de apoio que sustentam esse fluxo.

Na prática:

- o fluxo principal é composto por componentes conectados;

- os dados de apoio incluem filas, recursos, atributos, variáveis, coletores, contadores e outros objetos relacionados.

Essa distinção é central porque orienta tanto a modelagem quanto a extensão do software. Muitos erros de projeto ocorrem justamente quando se tenta tratar um elemento de apoio como se fosse componente do fluxo, ou vice-versa.

#### 8.4.1 Por que essa distinção melhora a arquitetura

Ao separar fluxo principal e dados de apoio, o GenESyS ganha clareza:

- o comportamento processual fica concentrado nos componentes;
- os objetos de suporte ficam representados como dados do modelo;
- a relação entre ambos pode ser expressa de forma explícita e controlada;
- o sistema se torna mais extensível.

Isso também melhora a legibilidade do software. O desenvolvedor consegue identificar com mais facilidade se um novo elemento que deseja implementar deve ser um componente, um dado do modelo ou uma combinação dos dois por composição ou agregação.

### 8.5 Composição, agregação e o desenho interno do modelo

A infraestrutura de `ModelDataDefinition` e `ModelComponent` revela que o GenESyS leva a sério a distinção entre composição e agregação.

Na composição, o elemento interno pertence estruturalmente ao objeto que o contém. Em termos práticos, isso significa que ele é criado, mantido e destruído como parte do ciclo de vida do objeto principal. Na agregação, ao contrário, há apenas referência ou vínculo com um dado existente no modelo, sem absorver seu ciclo de vida.

Essa distinção é particularmente útil quando um componente:

- mantém internamente contadores ou estatísticas próprias;
- ou referencia objetos externos como filas e recursos.

Sem essa separação, o sistema correria o risco de produzir vínculos mal definidos entre elementos do modelo, dificultando persistência, remoção, verificação e reuso.

#### 8.5.1 Internal data como materialização de composição

Quando um componente ou dado do modelo mantém *internal data*, ele está explicitando uma relação de composição. Isso costuma ser adequado para elementos que:

- existem apenas para apoiar o funcionamento interno daquele objeto;
- não fazem sentido como objetos independentes do modelo;
- devem desaparecer junto com o objeto principal.

### 8.5.2 Attached data como materialização de agregação

Quando um objeto mantém *attached data*, o que há é uma relação de agregação. O dado referenciado continua existindo no modelo como objeto próprio, podendo ser compartilhado, referenciado por outros elementos e persistido independentemente.

Essa separação entre composição e agregação torna a arquitetura do GenESyS muito mais limpa e semanticamente mais expressiva.

## 8.6 O sistema de *plugins*

A presença de *plugins* é uma das características mais importantes do GenESyS como software extensível. A própria classe *Simulator* já deixa isso evidente ao expor um *PluginManager*. Isso mostra que o mecanismo de *plugins* não é improvisado nem periférico; ele é parte estrutural do sistema.

Para o desenvolvedor, a consequência é clara: ampliar o GenESyS não significa necessariamente modificar diretamente o kernel. Em muitos casos, a estratégia mais adequada é introduzir novas capacidades por meio de *plugins*. Isso vale especialmente para:

- novos componentes;
- novos *data definitions*;
- novas extensões de linguagem;
- novos comportamentos especializados.

### 8.6.1 Por que *plugins* são importantes

Sem *plugins*, o crescimento do sistema dependeria de alterações cada vez mais profundas no núcleo. Isso tenderia a tornar o software mais rígido e mais difícil de manter. Com *plugins*, o projeto ganha:

- modularidade;
- extensibilidade;
- especialização por domínio;
- maior separação entre infraestrutura estável e funcionalidades adicionais.

Essa é uma decisão arquitetural especialmente apropriada para um simulador que pretende ser genérico e expansível.

### 8.6.2 Componentes e dados como extensões naturais

Do ponto de vista do modelo, muitos *plugins* se materializam justamente como novos componentes e novos dados. Isso significa que a arquitetura discutida até aqui *ModelDataDefinition*, *ModelComponent*, *internal data*, *attached data* não é apenas uma questão de elegância interna. Ela é o fundamento sobre o qual a extensibilidade do sistema se apoia.

Em outras palavras, compreender a arquitetura dos elementos do modelo é também compreender a arquitetura dos *plugins*.

## 8.7 Como pensar uma nova extensão

Um desenvolvedor que pretenda ampliar o GenESyS deve começar por uma pergunta muito simples:

O que exatamente estou tentando introduzir no sistema?

A partir dessa pergunta, costuma ser possível distinguir alguns casos:

- um novo comportamento no fluxo do modelo;
- um novo dado de apoio ao modelo;
- uma nova combinação entre comportamento e dados internos;
- uma nova necessidade de linguagem ou parser;
- uma nova aplicação ou ferramenta.

Se a necessidade for um novo comportamento no fluxo, a extensão provavelmente terá a forma de novo `ModelComponent`. Se for um novo objeto de apoio, provavelmente tomará a forma de novo `ModelDataDefinition`. Se envolver ambos, será necessário pensar cuidadosamente em composição, agregação e relações internas do novo elemento.

## 8.8 Leitura errada que este capítulo evita

Este capítulo procura evitar uma leitura errada muito comum: imaginar que o modelo do GenESyS seja composto apenas por caixas visuais na GUI. Na realidade, o modelo é uma estrutura muito mais rica, sustentada por uma hierarquia de classes e por uma distinção arquitetural clara entre comportamento e dados.

Também procura evitar outro erro: imaginar que *plugins* sejam apenas detalhes de carregamento dinâmico. No GenESyS, eles são parte da lógica de crescimento do sistema. A extensibilidade não é apêndice; é princípio arquitetural.

## 8.9 Considerações Finais

Este capítulo mostrou que o modelo do GenESyS é estruturado a partir de uma arquitetura coerente e extensível. A classe `ModelDataDefinition` fornece a infraestrutura base para elementos do modelo. A classe `ModelComponent` especializa essa base para representar comportamento no fluxo. A distinção entre fluxo principal e dados de apoio, bem como entre composição e agregação, organiza de forma clara a semântica interna do sistema. Finalmente, o mecanismo de *plugins* aparece como consequência natural dessa arquitetura, permitindo que o simulador seja ampliado sem perder coerência.

Tabela 8.1: Distinções centrais para compreender a estrutura do modelo no GenESyS.

| Distinção                              | Significado arquitetural                                                 |
|----------------------------------------|--------------------------------------------------------------------------|
| ModelComponent vs. ModelDataDefinition | Diferença entre comportamento no fluxo e dados/infraestrutura do modelo  |
| Fluxo principal vs. dados de apoio     | Diferença entre percurso processual e elementos que sustentam a execução |
| Internal data vs. attached data        | Diferença entre composição e agregação dentro do modelo                  |
| Kernel vs. <i>plugins</i>              | Diferença entre infraestrutura central e extensões especializadas        |

Com isso, o leitor já dispõe da base necessária para compreender como novos elementos podem ser introduzidos no sistema. O próximo passo natural é estudar o subsistema que conecta linguagem, expressões e semântica do modelo: o parser do GenESyS e sua relação com a linguagem interna do simulador.

Teste seu entendimento desse conteúdo respondendo as seguintes perguntas:

1. Qual é a vantagem arquitetural de fazer ModelComponent herdar de ModelDataDefinition?
2. Como a distinção entre fluxo principal e dados de apoio ajuda a compreender a estrutura do modelo?
3. Qual é a diferença entre *internal data* e *attached data* no desenho interno do GenESyS?
4. Por que o sistema de *plugins* deve ser entendido como parte estrutural da arquitetura e não apenas como mecanismo opcional de extensão?



## 9. Parser, expressões e linguagem interna

### 9.1 Introdução

Ao longo da primeira metade deste livro, a linguagem do *GenESyS* apareceu para o usuário principalmente por meio da aba *Simulang*. Naquela etapa, o interesse central estava em perceber que o modelo gráfico também possui uma representação textual. Para o desenvolvedor, porém, essa constatação é apenas o ponto de partida. O problema real agora é outro: como o sistema interpreta expressões, como a linguagem se liga ao estado do modelo e como novas capacidades sintáticas e semânticas podem ser introduzidas no simulador.

Este capítulo trata exatamente desse subsistema. Seu foco não está apenas na noção genérica de “parser”, mas no modo concreto como o *GenESyS* organiza a avaliação de expressões, o tratamento de símbolos do modelo, a integração com funções estocásticas, a gramática Bison/Flex e os pontos de extensão que permitem ao sistema crescer.

A importância desse tema é grande. Em um simulador como o *GenESyS*, a linguagem não é ornamento. Muitas decisões do modelo passam por expressões: parâmetros, fórmulas, referências a atributos, consultas a estatísticas, controles de simulação e funções probabilísticas. O parser, portanto, não é apenas uma conveniência textual; ele é parte da semântica operacional do simulador.

## 9.2 O papel do parser no GenESyS

O parser do *GenESyS* deve ser entendido como a infraestrutura que torna possível interpretar expressões e funções em contexto de modelo. Isso significa que ele não opera apenas sobre texto abstrato, mas sobre texto que precisa ser relacionado a elementos concretos do simulador:

- atributos da entidade corrente;
- controles e respostas de simulação;
- contadores e coletores estatísticos;
- variáveis, fórmulas, filas, recursos e outros objetos do modelo;
- funções matemáticas e probabilísticas;
- informações sobre o estado corrente da simulação.

Em outras palavras, o parser funciona como ponte entre linguagem e estado interno do sistema. Essa é a razão pela qual ele é tão importante na arquitetura do GenESyS.

### 9.2.1 Parser como subsistema semântico

É útil evitar uma leitura limitada do parser como simples analisador sintático. No *GenESyS*, ele faz mais do que reconhecer uma expressão bem formada. Ele também precisa:

- resolver nomes de símbolos;
- identificar a natureza desses símbolos;
- obter valores associados ao estado atual do modelo;
- integrar funções estocásticas apoiadas em um amostrador;
- propagar mensagens de erro úteis.

Isso significa que a leitura correta do parser no GenESyS é semântica e não apenas sintática.

## 9.3 A interface `Parser_if`

A interface `Parser_if` oferece uma síntese muito clara do papel do parser no projeto. Ela define um contrato mínimo para avaliação de expressões e funções estocásticas, deixando explícitas algumas responsabilidades centrais:

- avaliar expressões e retornar um valor numérico;
- oferecer uma variante que reporte sucesso e mensagem de erro sem lançar exceção;
- disponibilizar a última mensagem de erro;
- associar o parser a uma instância de `Sampler_if`;
- expor o driver interno do parser para cenários avançados.

Essa interface é importante porque mostra que o parser é visto, arquiteturalmente, como serviço do simulador e não como detalhe acidental de implementação.

### 9.3.1 Avaliação de expressões

O ponto mais evidente da interface é o método de avaliação de expressões. A existência de duas formas principais de `parse` é particularmente reveladora:

- uma forma que pode lançar exceções;
- uma forma que evita exceções e informa explicitamente sucesso e mensagem de erro.

Essa decisão melhora a integração do parser com diferentes camadas do sistema. Em alguns contextos, o desenvolvedor pode preferir tratamento por exceção. Em outros, especialmente em validação e interface de usuário, é mais conveniente receber o estado de sucesso de forma explícita.

### 9.3.2 Integração com `Sampler_if`

A interface também deixa explícito que o parser não é apenas analisador de expressões determinísticas. Ele integra funções estocásticas apoiadas em `Sampler_if`. Isso é importante porque reforça a natureza específica da linguagem do GenESyS: ela não é apenas uma mini-linguagem aritmética, mas uma linguagem voltada à simulação.

Em termos práticos, isso significa que funções probabilísticas da linguagem dependem da presença de um amostrador associado ao parser. Com isso, a semântica das expressões incorpora naturalmente mecanismos de geração aleatória.

## 9.4 A classe `Model` como usuária do parser

O papel do parser dentro do kernel fica especialmente claro na classe `Model`. Ela expõe métodos como `parseExpression` e `checkExpression`, o que mostra que o modelo usa o parser diretamente para:

- avaliar expressões definidas no contexto do modelo;
- validar expressões inseridas pelo usuário;
- verificar referências a *data definitions* no texto das expressões.

Esse detalhe é arquiteturalmente muito importante. Ele significa que o parser não está isolado em uma camada externa; ele participa da vida do modelo.

### 9.4.1 Por que `Model` invoca o parser

A presença desses métodos em `Model` revela um princípio importante: expressões fazem parte da semântica do modelo, e não apenas da interface. Um componente ou dado do modelo que armazena uma expressão precisa poder validá-la e, em muitos casos, avaliá-la dentro do contexto corrente da simulação.

Essa escolha arquitetural é muito boa porque mantém a linguagem conectada ao objeto ao qual ela realmente pertence: o modelo.

## 9.4.2 Expressões e verificação de consistência

Também é importante notar que a classe `Model` oferece `checkExpression`. Isso mostra que o parser participa não apenas da execução, mas também da fase de verificação do modelo. Em outras palavras, a validade de uma expressão é tratada como parte da integridade do modelo, e não como problema a ser descoberto apenas durante uma execução já iniciada.

Essa decisão melhora muito a robustez do sistema e ajuda a explicar por que a verificação do modelo é uma etapa tão importante no uso e no desenvolvimento do GenESyS.

## 9.5 A gramática Bison/Flex do GenESyS

A implementação concreta do parser em `source/parser/parserBisonFlex` mostra de forma bastante clara a maturidade desse subsistema. O arquivo `bisonparser.yy` define uma gramática C++ baseada em `lalr1.cc`, com classe de parser própria, tokens tipados, localizações, código de suporte e integração explícita com o `genesyspp_driver`. Isso mostra que o parser foi pensado como componente sério da infraestrutura do simulador, e não como rotina textual improvisada.

A gramática cobre, entre outros elementos:

- números;
- operadores relacionais e lógicos;
- funções trigonométricas e matemáticas;
- funções probabilísticas;
- funções associadas ao kernel;
- símbolos do modelo;
- extensões introduzidas por *plugins*.

Essa abrangência já basta para mostrar que o parser é um dos subsistemas de maior densidade semântica do projeto.

### 9.5.1 Uma linguagem realmente integrada ao modelo

A leitura da gramática mostra algo particularmente relevante: muitos tokens e produções não se referem apenas a construções sintáticas abstratas, mas a objetos reais do modelo. Há regras para atributos, respostas de simulação, controles de simulação e elementos como variáveis, fórmulas, filas e recursos. Também aparecem funções associadas a estatísticas, contadores e estado da simulação.

Isso reforça um ponto essencial:

A linguagem do GenESyS não é externa ao simulador; ela é uma projeção textual da semântica do modelo e da simulação.

### 9.5.2 Funções probabilísticas e amostragem

A gramática deixa explícita a presença de funções probabilísticas como exponencial, normal, uniforme, Weibull, lognormal, gama, Erlang, triangular e beta, entre outras. Em todas elas, a semântica operacional depende do `Sampler_if` associado ao parser.

Isso é muito importante para o desenvolvedor, porque mostra que:

- a linguagem do simulador incorpora diretamente amostragem estocástica;
- a implementação do parser precisa manter contrato consistente com o subsistema estatístico;
- o tratamento de erros nessas funções não é apenas sintático, mas também operacional.

## 9.6 Símbolos do modelo e dependência do estado corrente

Uma das características mais interessantes do parser do GenESyS é sua dependência explícita do estado corrente da simulação. A gramática mostra, por exemplo, que o acesso a atributos depende da entidade corrente associada ao evento atual. Da mesma forma, funções como `TNOW`, `TFIN`, número de repetições e identificadores da entidade corrente dependem do estado mantido em `ModelSimulation` e no evento atual.

Essa característica é arquiteturalmente forte porque revela que a interpretação de expressões no GenESyS é contextual. A mesma expressão pode ter significado diferente de acordo com:

- o modelo carregado;
- a entidade corrente;
- o evento em processamento;
- o instante simulado;
- os dados do modelo disponíveis naquele contexto.

### 9.6.1 Parser como consumidor do estado do kernel

Isso significa que o parser consome diretamente informações do kernel. Ele consulta:

- o modelo;
- a simulação;
- o evento corrente;
- o *data manager*;
- coletores e contadores;
- controles e respostas.

Essa dependência não é defeito. Pelo contrário, é parte da natureza do problema. Um parser para linguagem de simulação precisa conhecer o universo de símbolos e estados do simulador.

## 9.7 Pontos de extensão por *plugins*

A gramática do *GenESyS* revela de forma muito clara a existência de regiões pensadas para extensão. Os blocos marcados por comentários estruturados para inclusões, tokens, tipos e produções de *plugins* mostram que o sistema foi desenhado para crescer.

Esse aspecto é particularmente importante para o desenvolvedor. Ele mostra que a ampliação da linguagem não depende necessariamente de reescrita completa da gramática base. Há pontos arquiteturalmente previstos para que novos elementos sejam introduzidos.

### 9.7.1 Extensão sintática e semântica

Quando um *plugin* amplia a linguagem, ele não introduz apenas novos tokens. Em geral, ele introduz também:

- novos símbolos;
- novas funções;
- novas regras de interpretação;
- novas relações entre texto e dados do modelo.

Isso significa que a extensão do parser é simultaneamente sintática e semântica. É precisamente por isso que a infraestrutura de `ParserChangesInformation` e `ParserManager` existe no projeto.

## 9.8 A classe `ParserManager`

A classe `ParserManager` torna explícita a ambição arquitetural do *GenESyS* de sustentar evolução da linguagem. Sua interface prevê:

- geração de um novo parser a partir de mudanças;
- conexão de um novo parser ao sistema.

Embora o próprio cabeçalho indique que partes importantes desse fluxo ainda estejam em desenvolvimento, a existência dessa classe já é arquiteturalmente relevante. Ela mostra que o *GenESyS* não trata a gramática como artefato inteiramente estático. Há, no desenho do sistema, espaço para gestão explícita da evolução do parser.

### 9.8.1 O que isso significa para o desenvolvedor

Para o desenvolvedor, isso significa que trabalhar com linguagem no *GenESyS* não se limita a modificar um arquivo `Bison` ou `Flex`. O projeto prevê, em nível arquitetural, um subsistema responsável por:

- descrever mudanças;
- gerar artefatos de parser;
- conectar novos parsers ao simulador.

Mesmo que essa infraestrutura ainda esteja amadurecendo, ela já orienta a forma correta de pensar a evolução da linguagem.

## 9.9 Erros, mensagens e robustez

Outro aspecto importante da implementação do parser é o tratamento de erros. A gramática mostra preocupação explícita com mensagens úteis, inclusive no caso de tokens ilegais, literais não encontrados e falhas em funções probabilísticas. Isso é importante porque a linguagem de simulação, quando mal interpretada ou mal diagnosticada, se torna rapidamente fonte de frustração para usuário e desenvolvedor.

A robustez do parser depende, portanto, de três fatores:

- reconhecimento sintático correto;
- integração semântica correta com o modelo;
- produção de mensagens de erro suficientemente informativas.

Essa tríade é especialmente importante em um sistema em que expressões podem aparecer em muitos pontos do modelo.

## 9.10 Como estudar e modificar o parser de forma produtiva

A forma mais produtiva de estudar o parser do GenESyS é em camadas.

Primeiro, compreender a interface `Parser_if` e a forma como `Model` invoca o parser. Depois, estudar a gramática Bison/Flex e a maneira como ela acessa símbolos do modelo. Em seguida, identificar os pontos de extensão introduzidos por *plugins*. Só então faz sentido propor alterações mais significativas, especialmente quando envolvem novas funções, novos símbolos ou mudanças na gramática.

Também é importante lembrar que alterações no parser não são apenas alterações textuais. Elas podem afetar:

- verificação de modelos;
- semântica de componentes e *data definitions*;
- integração com amostragem;
- mensagens de erro;
- compatibilidade com *plugins*.

## 9.11 Considerações Finais

Este capítulo mostrou que o parser do *GenESyS* é muito mais do que uma camada auxiliar de texto. Ele é parte integrante da semântica do simulador. A interface `Parser_if`, a integração direta com `Model`, a gramática Bison/Flex, o uso de símbolos do modelo e da simulação, a

Tabela 9.1: Camadas de leitura do parser do GenESyS.

| Camada                  | Pergunta principal                                                     |
|-------------------------|------------------------------------------------------------------------|
| Interface               | Que serviço o parser oferece ao restante do sistema?                   |
| Integração com o modelo | Como <code>Model</code> e o estado da simulação utilizam esse serviço? |
| Gramática concreta      | Como a linguagem é reconhecida e avaliada?                             |
| Extensibilidade         | Como novos símbolos e funções entram no sistema?                       |
| Robustez                | Como erros são detectados e comunicados?                               |

dependência de um amostrador e os pontos explícitos de extensão por *plugins* revelam um subsistema arquiteturalmente denso e estrategicamente importante para o projeto.

Com isso, o leitor já dispõe dos elementos principais para compreender como o *GenESyS* liga linguagem, modelo e execução. O passo seguinte é ampliar novamente o foco, saindo do parser e voltando ao ecossistema mais amplo do projeto: aplicações, ferramentas, exemplos em C++, testes e estratégias de evolução do software. É exatamente isso que será tratado no próximo capítulo, que fecha esta versão do manual do desenvolvedor.

Teste seu entendimento desse conteúdo respondendo as seguintes perguntas:

1. Por que o parser do *GenESyS* deve ser entendido como subsistema semântico e não apenas sintático?
2. Qual é a importância de a classe `Model` expor métodos como `parseExpression` e `checkExpression`?
3. Como a gramática do *GenESyS* evidencia dependência do estado corrente do modelo e da simulação?
4. Por que os pontos de extensão por *plugins* são essenciais para a evolução da linguagem do simulador?



## 10. Applications, tools, exemplos em C++, testes e evolução do projeto

### 10.1 Introdução

Este capítulo encerra a estrutura principal deste manual do desenvolvedor ampliando novamente o foco. Nos capítulos anteriores, a atenção recaiu sobre a arquitetura central do sistema: organização do código-fonte, kernel, estrutura dos elementos do modelo, *plugins* e parser. Agora, o objetivo é examinar como essa infraestrutura se projeta em aplicações concretas, ferramentas auxiliares, exemplos programáticos em C++ e mecanismos de teste. Em outras palavras, este capítulo trata do ecossistema de desenvolvimento que se forma ao redor do núcleo do *GenESyS*.

A importância desse fechamento é grande. Um simulador extensível não vive apenas de suas classes centrais. Ele também depende de aplicações que o exponham ao usuário, de ferramentas que apoiem seu uso e sua manutenção, de exemplos que mostrem como a biblioteca pode ser utilizada programaticamente e de testes que sustentem a evolução segura do sistema. Sem esses elementos, o núcleo do software até poderia existir, mas o projeto como plataforma de desenvolvimento ficaria empobrecido.

Por isso, este capítulo deve ser lido como uma síntese final da segunda metade do livro. Ele mostra como o *GenESyS* transita:

- do kernel para as aplicações;
- da arquitetura para o uso programático;
- da extensão funcional para a manutenção técnica;
- da infraestrutura estável para a evolução futura do projeto.

## 10.2 Applications como projeções do kernel

A pasta `source/applications` é o lugar onde o GenESyS deixa de aparecer apenas como infraestrutura e passa a assumir formas concretas de interação. Essa observação é fundamental para o desenvolvedor. Ela mostra que o projeto não foi desenhado para existir apenas como biblioteca abstrata, mas para sustentar aplicações reais sobre o seu núcleo.

Arquiteturalmente, isso é muito valioso. Ao separar o kernel das aplicações, o sistema ganha:

- modularidade;
- reuso do núcleo em diferentes interfaces;
- menor acoplamento entre lógica de simulação e forma de interação;
- maior capacidade de evolução.

Na prática, essa separação permite que GUI, aplicação terminal e outras camadas futuras existam como projeções distintas do mesmo conjunto de serviços centrais.

### 10.2.1 A GUI como aplicação sobre o núcleo

Na primeira metade deste livro, a GUI foi tratada como o caminho principal de uso do simulador. Agora, ela passa a ser vista de outro modo: como aplicação construída sobre a infraestrutura do GenESyS. Essa mudança de perspectiva é especialmente importante para o desenvolvedor.

A GUI não é o núcleo. Ela é uma aplicação que:

- consome o modelo;
- consome a semântica dos componentes e dados;
- se conecta aos mecanismos de simulação;
- expõe essas capacidades ao usuário por meio de uma interface visual.

Essa separação é arquiteturalmente correta e explica por que o sistema pode, em princípio, sustentar outras aplicações além da interface gráfica.

### 10.2.2 A aplicação terminal como forma alternativa de uso

A árvore de build atual do projeto mostra que a aplicação terminal é tratada de forma particularmente estruturada. O `CMakeLists.txt` de `source/applications/terminal` explicita o uso de uma aplicação chamada `GenesysShell`, bem como a possibilidade de selecionar outras classes de aplicação de terminal e até arquivos de exemplo específicos em

source/applications/terminal/examples para compor o executável terminal :contentReference[oaicite:0]index=0

Isso é extremamente importante para o desenvolvedor. Mostra que o terminal não é apenas ferramenta auxiliar antiga ou caminho marginal de uso. Ele é uma aplicação efetiva do GenESyS e, mais do que isso, um ambiente valioso para:

- estudar o uso programático do simulador;
- construir exemplos compactos;
- desenvolver aplicações leves sobre a biblioteca;
- verificar comportamentos sem a mediação da GUI.

## 10.3 Os exemplos em C++ como material de desenvolvedor

Na primeira metade do livro, os exemplos em C++ da aplicação terminal foram deliberadamente deixados de lado, porque o manual do usuário não deveria começar por código. Agora, porém, eles passam a ocupar um lugar central. Isso ocorre porque esses exemplos mostram uma das capacidades mais importantes do GenESyS como software: seu uso como biblioteca de simulação.

O `CMakeLists.txt` da aplicação terminal mostra explicitamente a existência de exemplos seleccionáveis por build, inclusive exemplos *smart* e exemplos mais elaborados de ensino, o que confirma o papel desses arquivos como material de estudo e demonstração arquitetural :contentReference[oaicite:1]index=1

### 10.3.1 Por que esses exemplos são tão valiosos

Os exemplos em C++ são valiosos porque permitem observar o simulador por outro ângulo. Na GUI, o modelo aparece como artefato visual e operacional. Nos exemplos programáticos, ele aparece como objeto construído explicitamente pelo desenvolvedor. Isso ajuda a revelar:

- como o *Simulator* é instanciado;
- como o modelo é construído por código;
- como componentes e *data definitions* são criados e relacionados;
- como a execução é disparada;
- como a biblioteca do simulador pode ser reutilizada em novas aplicações.

### 10.3.2 A ponte entre exemplos em C++ e modelos salvos

Uma das características mais didaticamente ricas do projeto é a possibilidade de relacionar:

- exemplos construídos programaticamente em C++;
- modelos salvos na pasta `models/`.

Essa ponte é extremamente útil para o desenvolvedor porque mostra o mesmo sistema sob duas perspectivas:

- como objeto de biblioteca;
- como artefato de modelagem e uso.

Essa correspondência ajuda a consolidar a compreensão do GenESyS como plataforma, e não apenas como aplicação pronta.

## 10.4 Tools como camada de apoio

A pasta `source/tools` representa uma região importante do projeto, ainda que frequentemente menos comentada do que `kernel`, `parser` e *plugins*. Sua função geral é apoiar uso, desenvolvimento, integração ou manutenção do sistema.

Do ponto de vista do desenvolvedor, ferramentas são relevantes por três razões:

- ajudam a tornar o ecossistema do simulador mais produtivo;
- oferecem pontos de suporte à observação ou automação;
- podem servir como base para novas integrações.

Em termos arquiteturais, a existência de `tools` mostra que o projeto não está limitado à dicotomia “kernel versus aplicação”. Há também espaço para instrumentos intermediários, utilitários e mecanismos de apoio.

### 10.4.1 Quando estudar `tools`

Nem sempre as ferramentas devem ser a primeira prioridade de leitura. Em geral, faz mais sentido estudá-las depois que o desenvolvedor já compreendeu:

- o núcleo do simulador;
- o sistema de *plugins*;
- o `parser`;
- ao menos uma aplicação concreta, como a GUI ou a terminal.

Depois disso, a leitura das ferramentas tende a ser muito mais produtiva, porque o desenvolvedor consegue enxergar melhor sua função no ecossistema do projeto.

## 10.5 A importância dos testes

Em um projeto do porte e da ambição do *GenESyS*, a existência de testes é parte essencial da arquitetura evolutiva do software. Isso não significa apenas que o projeto “tem testes”, mas que o próprio sistema de build já os trata como parte estruturada do desenvolvimento.

O `CMakeLists.txt` de `source/tests` mostra explicitamente a divisão entre testes unitários e *smoke tests*, com um alvo agregado `genesys_tests` e a possibilidade de habilitar ou desabilitar a construção dos testes de fumaça por opção de build `:contentReference[oaicite:2]index=2`

Essa organização é particularmente valiosa porque dá ao desenvolvedor um modo de pensar a validação em camadas:

- testes unitários para comportamento local e invariantes menores;
- *smoke tests* para sanidade mais ampla do sistema.

### 10.5.1 Por que testes são parte da arquitetura e não só da rotina

É comum pensar testes apenas como atividade externa ao projeto principal. No GenESyS, porém, eles devem ser lidos de forma mais forte: como parte da infraestrutura de evolução do sistema. Isso é particularmente importante porque o software tem:

- kernel relativamente denso;
- mecanismos de extensão;
- múltiplas aplicações;
- integração entre parser, modelo, eventos e observabilidade.

Em um contexto assim, mudar código sem apoio de testes tende a ser arriscado. Portanto, o desenvolvedor deve ver a pasta `source/tests` como aliada direta na manutenção do projeto.

### 10.5.2 Como o desenvolvedor deve usar os testes

Uma prática madura de desenvolvimento no GenESyS deve considerar os testes em pelo menos três momentos:

1. antes da alteração, para compreender a base atual de comportamento;
2. durante a alteração, para evitar regressões locais;
3. depois da alteração, para validar integração e sanidade geral.

Essa disciplina é especialmente importante quando a mudança atinge kernel, parser, *plugins* ou aplicações.

## 10.6 Build, aplicações e seleção de comportamentos

Ainda que o build não seja o eixo norteador do livro, ele continua oferecendo ao desenvolvedor um conjunto importante de sinais sobre a organização do projeto. O arquivo de build da aplicação terminal, por exemplo, mostra que o sistema foi pensado para permitir seleção configurável do comportamento do executável terminal, seja por classes específicas, seja por exemplos concretos em `source/applications/terminal/examples`:[contentReference\[oaicite:3\]index=3](#)

Isso revela algo importante sobre o projeto:

O GenESyS não é apenas um programa; ele é uma plataforma sobre a qual comportamentos de aplicação podem ser escolhidos e montados.

Essa observação interessa muito ao desenvolvedor porque reforça a modularidade do sistema e ajuda a pensar futuras aplicações sobre o núcleo.

## 10.7 Evolução do projeto e caminhos de contribuição

A leitura combinada de kernel, parser, *plugins*, applications, tools e tests mostra que o GenESyS é um projeto em evolução contínua. Essa evolução pode ocorrer em várias frentes:

- aperfeiçoamento do kernel;
- ampliação do conjunto de componentes e *data definitions*;
- evolução da linguagem e do parser;
- fortalecimento da GUI e de outras aplicações;
- ampliação de ferramentas auxiliares;
- consolidação da base de testes.

Do ponto de vista do desenvolvedor, a pergunta mais útil deixa de ser:

Onde eu posso mexer?

e passa a ser:

Em que camada do sistema minha alteração faz mais sentido arquiteturalmente?

Essa mudança de perspectiva é o que distingue uma modificação oportunista de uma contribuição tecnicamente madura.

### 10.7.1 Como escolher um ponto de entrada para contribuir

A escolha do ponto de entrada depende muito do tipo de contribuição desejada.

- Se a intenção for alterar a dinâmica central da simulação, o estudo deve começar no kernel.
- Se a intenção for ampliar linguagem e expressões, o estudo deve começar no parser.
- Se a intenção for acrescentar elementos de modelagem, a leitura deve concentrar-se em *plugins*, componentes e *model data*.
- Se a intenção for melhorar a experiência de uso, a GUI e as aplicações tornam-se centrais.
- Se a intenção for aumentar robustez, testes e tracing passam a ser prioridade.

Essa forma de pensar melhora bastante a qualidade da contribuição ao projeto.

### 10.7.2 O papel dos exemplos e testes na evolução

Exemplos e testes desempenham papéis complementares na evolução do software.

- Os exemplos mostram como usar o sistema e ajudam a explorar novas capacidades.

- Os testes ajudam a verificar se o sistema continua correto ao ser modificado.

Um desenvolvedor maduro do GenESyS precisa valorizar ambos. Sem exemplos, o sistema perde capacidade de demonstração e aprendizado. Sem testes, perde segurança evolutiva.

## 10.8 Uma visão final integrada do GenESyS

Chegando ao final desta versão do manual do desenvolvedor, é útil voltar à visão integrada do projeto.

O *GenESyS* pode ser compreendido como:

- um kernel de simulação;
- uma hierarquia de objetos que representam modelo, componentes e dados;
- um sistema de linguagem e parser;
- uma infraestrutura de extensibilidade por *plugins*;
- um conjunto de aplicações construídas sobre o núcleo;
- um ecossistema de exemplos, ferramentas e testes que sustenta uso e evolução.

Essa visão integrada é importante porque evita leituras parciais demais. O GenESyS não é apenas GUI, não é apenas parser, não é apenas *plugin framework*, nem apenas biblioteca de simulação. Ele é a articulação de todas essas dimensões.

Tabela 10.1: Dimensões principais do GenESyS como plataforma de software.

| Dimensão     | Papel no projeto                                                    |
|--------------|---------------------------------------------------------------------|
| Kernel       | Infraestrutura central da simulação                                 |
| Parser       | Linguagem, expressões e semântica textual do modelo                 |
| Plugins      | Extensibilidade do simulador e ampliação dos elementos de modelagem |
| Applications | Formas concretas de uso, como GUI e aplicação terminal              |
| Tools        | Apoio ao uso, integração e manutenção                               |
| Tests        | Segurança evolutiva, validação e prevenção de regressões            |
| Examples     | Material de demonstração, estudo e uso programático da biblioteca   |

## 10.9 Considerações Finais

Este capítulo fechou a estrutura principal do manual do desenvolvedor mostrando que o *GenESyS* não deve ser lido apenas por suas classes centrais, mas também por seu ecossistema de aplicações, ferramentas, exemplos e testes. A GUI aparece agora como aplicação sobre o núcleo. A aplicação terminal e seus exemplos em C++ revelam o uso do simulador como biblioteca. As ferramentas ampliam o ambiente de desenvolvimento. A árvore de testes fornece

base para evolução segura. E a organização geral do projeto mostra que o GenESyS deve ser pensado como plataforma de software, e não apenas como simulador pronto.

Com isso, o leitor já dispõe de uma visão suficientemente ampla e tecnicamente fundamentada para usar, estudar e ampliar o GenESyS de forma madura. A partir daqui, o aprofundamento pode seguir diferentes trilhas: revisão e melhoria da GUI, estudo mais detalhado de componentes e *plugins*, evolução do parser, fortalecimento da infraestrutura de testes, desenvolvimento de novas aplicações ou criação de novos domínios de modelagem sobre o mesmo núcleo. Em todos esses casos, a lógica permanece a mesma: compreender a camada correta, respeitar a arquitetura do sistema e fazer a evolução do simulador a partir de seus próprios princípios estruturais.

Teste seu entendimento desse conteúdo respondendo as seguintes perguntas:

1. Por que a GUI deve ser entendida, do ponto de vista do desenvolvedor, como aplicação sobre o núcleo e não como o sistema inteiro?
2. Qual é o valor arquitetural dos exemplos em C++ da aplicação terminal?
3. Por que a base de testes do projeto deve ser tratada como parte da infraestrutura de evolução do software?
4. Como a distinção entre kernel, parser, *plugins*, applications, tools, examples e tests ajuda o desenvolvedor a escolher melhor o ponto de entrada de uma contribuição?



## Bibliography

RASCUNHO